

1주차

메인 교과서 : Russell의 Artificial Intelligence : A Modern Approach

<https://aima.cs.berkeley.edu/>

1주~7주 : 실생활 문제 Real-life Problem, Optimization Problems >> AI Searching 알고리즘

Solution의 형식이 Exact, Precise하지 않음.

Real-time, Superfast, Suboptimal(optimal에 근접한)

9-15주 : 인공지능, 머신러닝, 딥러닝

예측, 분류, 인식, 데이터사이언스

정확도(Accuracy) 정말도 집중 항상 기술

AI의 정의

AI 시스템이란?

1997 IBM 딥블루

1950년대에 인공지능이 시작

정의 :

- | | |
|----------------------------------|---------------------|
| 1. System that think like Humans | 인간처럼 생각하는 시스템 |
| 2. System that think Rationally | 이성적(합리적)으로 생각하는 시스템 |
| 3. System that act like Humans | 인간처럼 행동하는 시스템 |
| 4. System that act Rationally | 이성적으로 행동하는 시스템 |

1. Acting Humanly 사람처럼 행동함

Alan Turing 앨런 튜링(1950) : “튜링 테스트”

중국어 방 논증

-> Can machine think/behave/act intelligently?

사람처럼 지적으로 생각/행동할 수 있을까?

문제 : 튜링 테스트는 NOT Reproducible and Mathematical Analysable 재현과 재상산이 불가능!

수학적 증명이 없기 때문..

2. Thinking Humanly 사람처럼 행동함

인간 두뇌에 대한 “과학적 이론”이 필요함.

knowledge 지식, 신경회로가 어떻게 구성되는가?

어떻게 validate 할 것인가?

1) Cognitive Science 인지과학

Top-down 접근

심리학 실험, 동물실험

실제 인간에 대한 행동들을 테스트

2) Cognitive Neuroscience 뇌과학

Bottom-Up 접근. Data-driven

EEG 뇌파검사 등 neurological/bio signal 생체 데이터를 통해

문제 : Thinking Humanly is NOT

- 이 둘은 더이상 AI 분야가 아니다!

- 이론들의 설명력이 부족하다. human general intelligence 설명 명쾌하지 않음.(?)

3. Thinking Rationally 이성적으로 생각함

Law of Thought 생각의 법 (by 아리스토텔레스) : 무엇이 맞는(Correct) 생각의 프로세스인가?

고대 그리스부터 다양한 논리(logics)이 발전됨.

logic 논리 : notation and rules of thoughts 생각에 대한 표시와 규칙

=> Syntax

문제 : 모든 지능적 행동이 아직까지 논리(의미/형식/문법)로 설명할 수 없다.

4. Acting Rationally 이성적으로 행동함

Think, Human을 정의내리기 어려움
그래서 Act, Ration이라는 쉬운 정의를 채택!

Rational Behavior 이성적 행동 : doing the Right thing. 올바른 것을 행동하는 것

> Right thing : expected to maximize GOAL achievement, given(under) the available information
주어진 정보 안에서 GOAL을 최대화시키도록 하는 것

문제 : Computational limitations(resource) make perfect rationality unachievable (NP-hard 문제)

계산적인 한계(자원; 현실적인 한계) 때문에 완벽하게 이성적인 것은 찾을 수 없다.

>>주어진 machine을 가지고 최선(best)의 선택을 하려고 하는 것이 현존하는 AI 시스템.

Agent : an entity that perceives and acts 실제 무언가를 인식하고 행동하는 개체

->이 과목의 대부분의 내용은 Agent를 설계하는 것

∴ rational agent == AI system == AI SW/프로그램/플랫폼

수식 $f = P \rightarrow A$ P : 프로그램 A: Agent

Function from percept history to actions 이제까지 감지해왔던 히스토리를 바탕으로 옳은 행동을 함.

★Agent program runs on the physical machine to produce f.

★Agent 프로그램은 f를 생성하기 위해서 물리적인 기계 위에서 돌아감.

-> for any tasks and environments(specified), we seek the agent(AI system/program/sw/alg) with the "best" performance.

어떤 작업이나 환경에서 가장 좋은 성능(GOAL을 최대화하는)을 내는 Agent를 찾아야 한다! 만들어야 한다!

영화 Matrix에서 Neo와 Agent Smith(인공지능 프로그램 Smith) 가 싸움.

예) Vacuum Cleaner Agent 청소기 Agent

perception : (input) 위치, 물질(먼지)

action : 왼쪽, 오른쪽, 빨아들여라..

Right Function이란? Rationality란?

doing the right == action rationally == rational agent

★★ A rational agent chooses whichever action maximizes the "expected" value of the "performance measure" given the percept sequence.

Rational Agent는 주어진 입력(Percept sequence)를 통해서 GOAL을 최대화 시키려고 하고, 이를 측정하기 위해서 Performance measure을 하는데, 이 점수화된 값을 최대화하려고 함. 수학적, 공학적, 구현가능한 의미를 포함한다.

VC Agent를 위한 Performance measure는 뭐가 있을까?

- 움직일 때마다 -1점
- 빨리 마무리하면 +1점
- 남은 먼지량에 따라서 - 마이너스
- 배터리 소모량 등등.....

결론적으로 Rational Agent는 완벽한 AI가 아니다!!

1. rational != omniscient(전지전능한) 이성적이라는 것은 전지전능한 것이 아니다!
: perception may not supply all information Agent가 모든 정보를 받아들이지 않았을 수 있다.
2. rational != clairvoyant(천리안) 이성적이라는 것은 천리안이 아니다.
: action(outcomes) may not be as expected Agent가 예상치 못한 결과를 내뱉을 수도 있다.

PEAS(task environment) specified

P : Performance measure

PEAS가 고려되어야 한다.

E : Environment

A : Action/Actuators

S : Sensors

e.g. 구글 무인 자동차

P : 안전성, 편안함, 가격, 준법성...

E : 도로, 고속도로, 날씨, 교통상황...

A : 스티어링, 브레이크, 엑셀...

S : GPS, 엔진 센서, 속도...

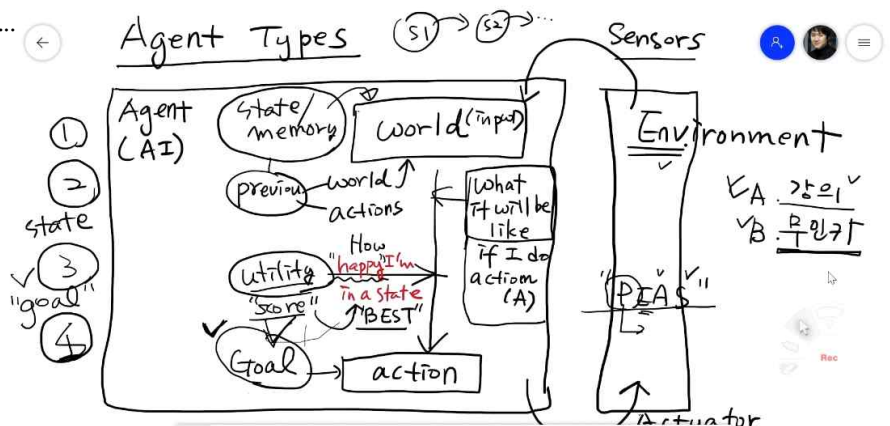
e.g. 인터넷 쇼핑 추천 시스템

P : 품질, 가격...

E : WWW, ...

A : 사용자에게 보여주는 화면

S : XML 페이지...



Agent Types Agent의 종류

Simple reflex agents

단순 반사 에이전트

Reflex agents with state(memory)

기억을 가진 반사 에이전트

Goal-Based agents

목표 기반 에이전트

Utility-based agents

효용 기반 에이전트

utility $\approx$ performance measure

1. Performances on AI Definitions.

4가지 AI 정의 특성을 교과서, 전통적 학술적 의미의 정의를 검토함.

하지만, 현 시점에서는 개념이 모호하고 경계선이 불분명한 이슈가 있음.

학문단위 융합, 통합 다학제 시대이므로 CS/AI/X + 인지과학/뇌과학 다양한 것들이 결합됨.

★ 나 자신만의 AI 정의를 확보하는 것이 중요. (우선 순위를 정해봐라!)

2. Turing Test

2014 BBC 유진 프로그램 공식적으로 Turing Test 통과

2018 구글 duplex : AI Assistant Call 사람같은 챗봇

2023 보스턴 다이내믹스 아틀라스 로봇 인간같이 행동

2023 ChatGPT vs Turing Test Turing test는 outdated 된건 아닐까..?

★ 나의 관점에서 어떠한 평가를 통해 AI를 새롭게 평가, 개선하는 고민을 해 보라!

3. PEAS : 무인자동차를 통해

Waymo 중심(영상)

PEAS의 실질적인 투영을 통해서 AI에 대한 정의, Turing Test에 대해서 고민을 해 보라!

2주차

"Problem Solving"

학습 목표

- * Problem-solving agents
- * Problem types
- * Problem formulation
- * Example problems
- * Basic Search Algorithm

CS(Computer Science)의 정의 : the science of intelligence 지능에 대한 과학

Algorithm의 정의 : a step-by-step procedure(CS 문제에 대한 solution) 순차적인 절차, 해결책

AI의 정의 : doing the "right thing" 옳은, 알맞은 일을 하는 것.

right thing이란 무엇인가? -> 공리주의 utility theory " 최대다수의 최대행복"

인공지능에서 doing the right thing은 무엇인가?

=> **Maximize Goal such that condition/constraints** 조건과 제약조건 하에 Goal을 최대화하는 것

↳ input/environment

어떻게 right thing을 측정할 것인가?

- performance measure(score func 점수함수, utility func 효용 함수, evaluation func 평가 함수)
- 누가 performance measure를 정하는가? 제시하는가? : 사용자, 발주자/개발자/구매고객이 정한다!

Problem-Solving

인공지능은 Problem-Solving을 어떻게 할까?

e.g. 주말에 루마니아에 있다. 현재 Arad라는 도시에 있다. 내일 비행기를 타고 Bucharest까지 가야 한다.
어떻게하면 A에서 B까지 잘 갈 수 있을까?

1. **Goal 목표** : B도시에 도착하기
2. **Performance measure 성과 측정** :
3. **Solution 해결책** : A에서 B까지 가는 방법
4. **Formulation 정형화, 수식화** :

<이걸 외울 필요는 없다.

Problem Formulation 문제 정형화란?

Problem은 네가지로 나누어진다.

1. **initial state(input) 초기 상태**
2. **successor function**: set of action-state pairs(ai) input을 받아서 지속적으로 뭘 할건가?
3. **goal test** 목표에 도달했는지 테스트
4. **path cost** : performance measure 성능 지표

Solution?

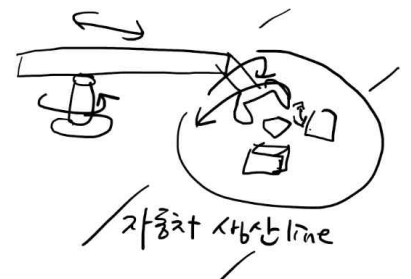
: **a sequence of actions** leading from the initial state to a goal state
초기부터 종료까지 이어지는 행동의 시퀀스

루마니아 Problem formulation

1. initial state: $x = \text{"at Arad"}$ Arad에 있는 상태
2. successor function: $S = \text{Arad} \rightarrow \dots \rightarrow \dots \rightarrow \text{Bucharest}$
3. goal test: "at Bucharest Bucharest에 있는 상태"
4. path cost: sum of distance of S 함수 S의 거리의 합

청소기 Problem formulation

1. initial state: int 형식의 먼지, 숫자로 표현된 robot의 위치
2. successor function: 왼쪽으로, 오른쪽으로, suck, stay 등의 sequence
3. goal test: 먼지가 없는 상황(왼쪽, 오른쪽 방)
4. path cost $\sim$ performance measure: Action당 1점... 등등



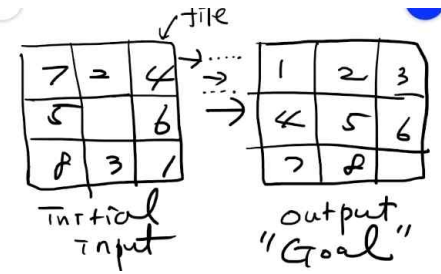
로봇 조립하기 Problem formulation

로봇팔로 자동차를 만드는 과정

1. initial state: 실수값으로 표현된 부품, 장비 등등의 좌표값
2. action($\sim$ s.f.): motion moves와 관련된 모든 작업 시퀀스
3. goal test: 조립 완성
4. cost: 실행 시간, reliable 신뢰도, 완성도

8-퍼즐 Problem formulation

1. initial state: 초기 상태
2. action(~= s.f.): 타일을 움직여 나가는 액션
3. goal test: 완성된 상태
4. cost: Time~Frequency 누가 더 빨리? 덜 움직?



Problem Solving 문제 해결

Def. : How to achieve the goal state through the very complex state space from the initial state?

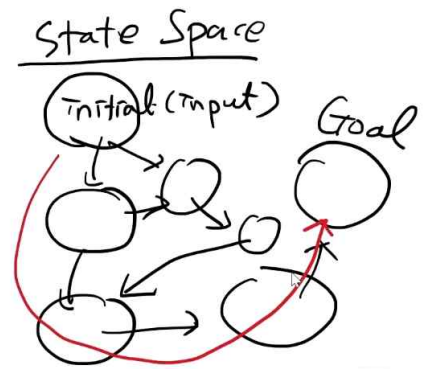
Initial state에서 시작하여 복잡한 상태를 거쳐서 어떻게 goal state를 달성할 것인가?

=> CS에서 Tree Search 트리 탐색이다!

자료구조와 알고리즘

Idea : Explore the state space by generating sequences of already-explored states(expanding state)

이미 확인한 상태를 기반으로 다음 상태를 줄시어 생성하면서 영역을 넓혀 나감.



문제 해결을 위한 탐색 기술.

Basic Search for Problem Solving

State vs Node

State 상태 : Conceptual Representation of a physical config.

물리적인 상태에 대한 개념적(추상적) 표현

Node 노드 : parent/child/link - tree

Search 탐색

defined by picking the order of "node expansion/visit"

'무엇'을 선택하는지에 따라 정의됨

- initial state == sinitial parent node

- **관점** : 완전함, 시간/공간 복잡도, 최적인가(optimality)

-> **BFS**(넓이우선탐색), **DFS**(깊이우선탐색)

기계적 탐색 알고리즘 탐색

다양한 기초 트리탐색의 알고리즘들이 존재함. 알고리즘 간의 비교를 분석한다.

BFS vs DFS 기초지식이 필요함

DLS vs IDS

DLS Depth-Limited Search

: DFS의 단점을 보완. 깊이 제한(L)이 있는 DFS

Depth Limit을 1, 2, 3 등으로 깊이를 제한하여 DFS를 함.

IDS Iterative Deepening Search

DLS의 단점을 보완하기 위해 등장. DLS를 수행하되 제한 깊이를 하나씩 올려나감.

function IDS(...)

for **depth** = 0 to infinite do

result = Depth-Limited Search(**depth**)

if(condition, result==goal)

end

IDS의 특징

시간복잡도 : b^d (b : 자식이 몇개? 통상적으로 2개 d : 트리기 depth 높이)
공간복잡도 : $b \cdot d \rightarrow$ optimize 가능하다. 최근에는 메모리가 커서 크게 의미 없음
complete? 완전한가? : YES
optimal? 최적인가? : YES

Traditional Search vs Informed Search(AI)

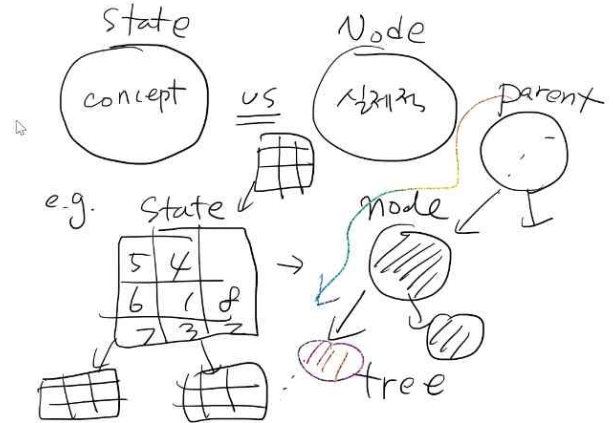
Informed search는 uninformed search(BFS, DFS, IDS등...) 보다 더 효율적으로 solution을 찾을 수 있다.

"Informed" : Problem-specific 지식, 입력값

Best-first Search Algorithm

: 각각의 노드에 대해 "evaluation function"을 사용함

1. Estimate/Predict the "desirability/potential" of each node
어떤 노드로 가면 유망할까? 매력적일까?를 추정/예측
2. Expand/visit most desirable unexpanded node
아직 확장되지 않은 노드 중 가장 유망한 노드 방문/확장
e.g. Greedy Algorithm, Branch-and-Bound, Backtracking, A* 알고리즘



Greedy Algorithm (search)

우리는 **Evaluation Function**이 필요하다! :

Heuristic function $h(n)$ $\rightarrow$ 정답을 빨리 찾기 위해 사용하는 **경험적이고 근사적인 판단 기준**

$h(n)$: **estimate of cost** from 'n' to the closest goal. 현재 노드 n에서 목표까지 드는 데 예상 비용

$h(n)$: straight-line distance from n to Bucharest (여기서는 B까지 직선 거리가 주어져있다고 가정하자)

특징

시간복잡도 : b^m (b : 자식이 몇개? m : 공간의 최대 깊이)

공간복잡도 : b^m

complete? : NO or YES Greedy는 완전하지 않음

optimal : No (항상 optimal한 것은 아니다.)

장점: 예측이 잘 맞는다면 탐색 속도가 엄청나게 빠릅니다.

단점: "최단 경로(Optimal Solution)"를 보장하지 않습니다. 당장 눈앞의 이익(직선거리)만 쫓다가 전체적으로 더 먼 길로 돌아갈 수 있기 때문입니다.

Baseline performance

A* Algorithm (A* Search)

: avoid expanding paths(nodes) that are already (known) expensive

이미 비싼 것으로 알려진 경로는 더 이상 확장하지 않는다.

$\rightarrow$ Cut off, Pruning-out 가지치기, bound (등등.. 다양한 용어)

불필요한 탐색은 하지 않음.

유사한 알고리즘

Backtracking : DFS with bound function

Bound function을 사용해 가망없는 경로를 안가는 DFS

Branch and bound B&B : BFS with bound function 모든 자식을 먼저 보고, Bound function으로 가지치기

A* 알고리즘 : Best-first B&B라고 불리며, 단순 너비 우선이 아니라 Best(가장 유망)한 것부터 가지치기

evaluation function : $f = g + h$

$g(n)$: cost so far to reach n from the initial node 시작 노드에서 현재노드 n 까지 오는데 실제로 든 비용

$h(n)$: estimated cost to goal node from n 현재 노드 n 에서 목표 노드까지 가는데 들 것으로 예상되는 비용

$f(n)$: estimated total cost through n to goal from initial 처음부터 목표까지 n 을 통해서 가는 예상 비용

A* 알고리즘은 "Admissible"(허용가능한, 절대로 실제 비용보다 더 높게 예측하지 않는) Heuristic을 쓴다. => "Optimal"

1. $h \leq h^*$ (h^* : true cost 실제 값. h : 예측값) h 를 실제값(h^*)보다 같거나 작게 해야.

2. $h \geq 0$

h 가 h^* 에 가까울수록 좋음.

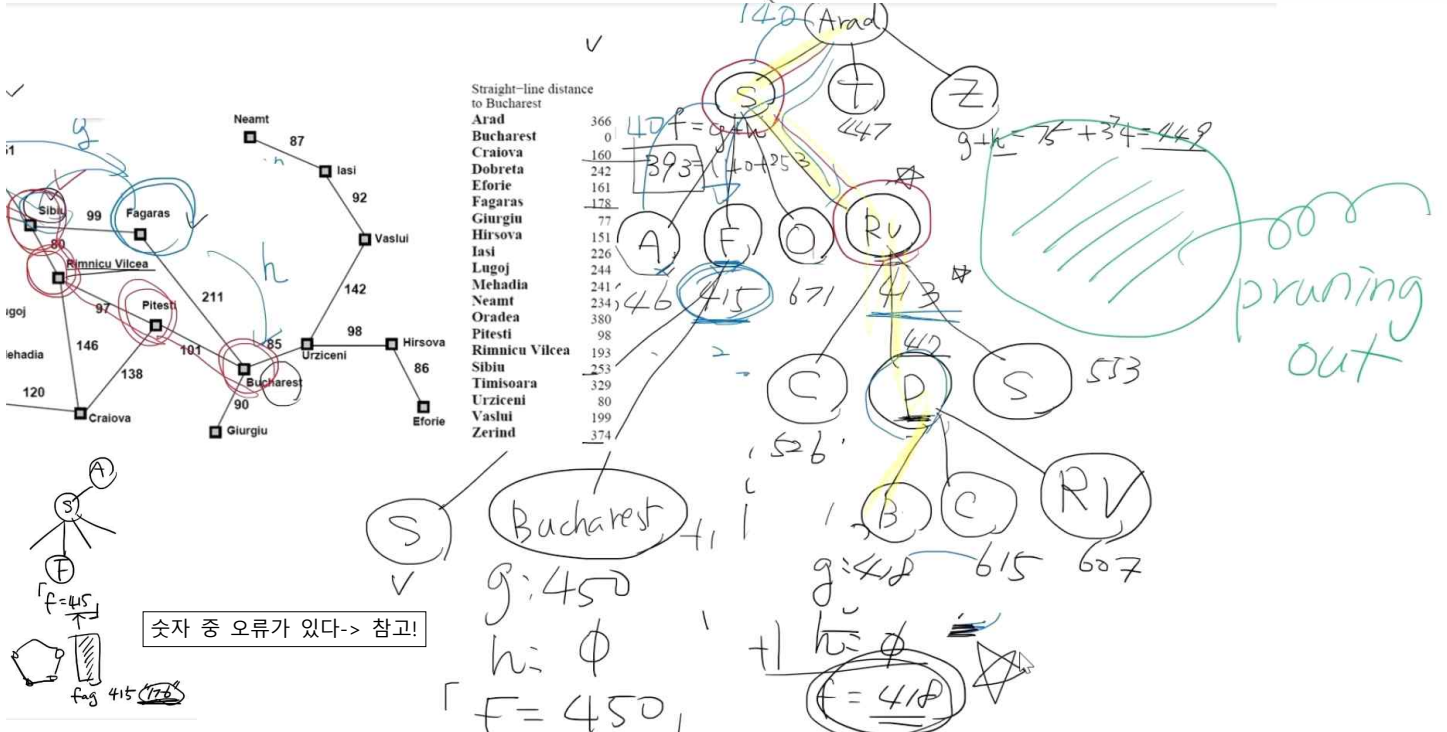
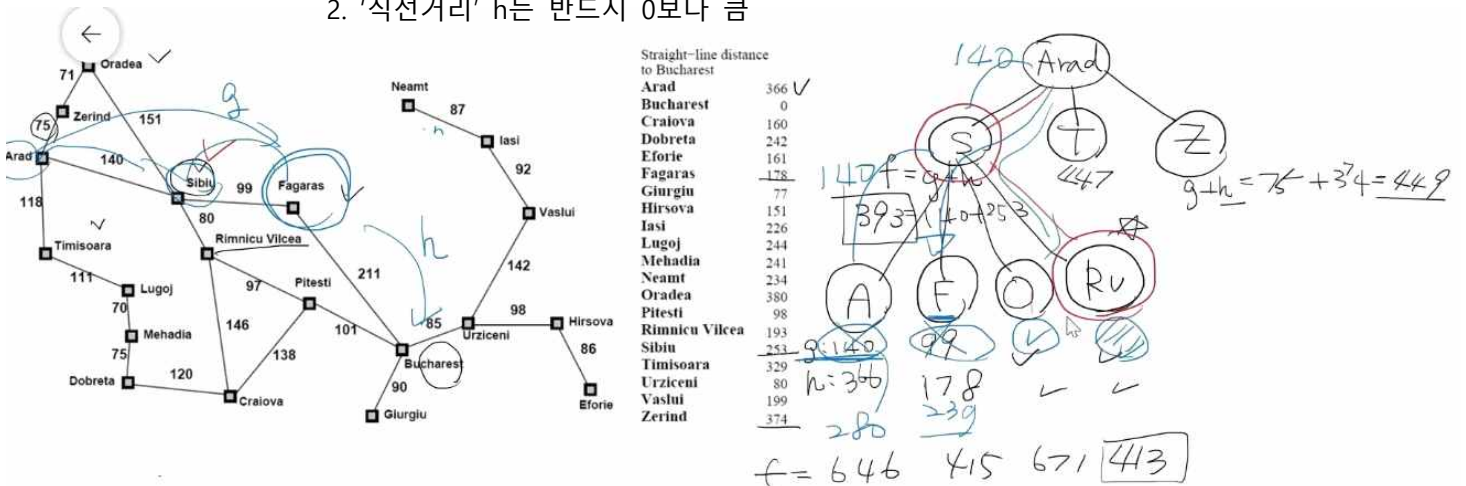
==> Minimize optimization problem

$f(n)$ 을 최소화하는 노드를 선택해나가는 과정. 가지치기 방식

Q : Arad-Bucharest에서 h 를 어떻게 설정할까?

A : 직선거리. 왜?? -> 1. '직선거리'는 Optimal 거리보다 반드시 작거나 같을 수밖에 없음

2. '직선거리' h 는 반드시 0보다 큼



e.g. remember 0/1 Knapsack problem

$f = g + h$ // h (fractional Knapsack)

못 넣은 것은 물건을 잘라서라도 집어넣는다.

g = 실제로 지금까지 담은 것

1. $h \geq h^*$ (h^* : true cost) h 를 h^* 보다 크거나 같게 해야.

==> Maximize optimization problem

$f(n)$ 을 최소화하는 노드를 선택해나가는 과정. 가지치기 방식

***참고 : Branch and Bound 분기 한정법**

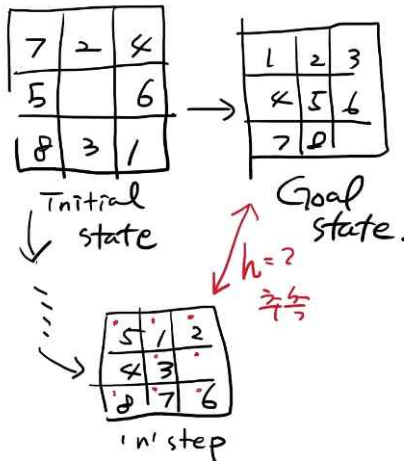
Branch 분기 : A*에서 현재 노드에서 갈 수 있는 다음 노드들을 펼쳐나가는 과정
 Bounding 한정 : A*에서 $f=g+h$ 값이 일종의 하한선 역할을 함.

어떤 경로의 f값이 이미 현재까지 찾은 최선의 경로보다 크다면, 그 쪽은 갈 필요가 없음 => Pruning
 이 때, $h \leq h^*$ 조건이 있어야만 안전하게 pruning 가능.

그래서...

g : exact/precise 정확한 값. 사람에 따라서 다르지 않다.
 h : 알고리즘의 성능을 좌우함

- complete한가? => YES
- 시간복잡도 : 확장 expansion의 순서가 고정되어 있지 않음..
 잘못된 h가 있다면, exponential하게 시간이 늘어날 수 있음.
- 공간복잡도 : 모든 노드를 메모리에 데이터를 가지고 있어야 함.
- optiaml한가? : YES



e.g. A*알고리즘을 사용한 8-Puzzle

h를 어떻게 추측할까? -> $f = g + h$

g : 처음 상태부터 지금까지 옮긴 (정확한) 횟수
 h : h를 어떻게 할 수 있을까?

- $h1(n)$ = **number of misplaced tiles** 잘못 놓여진 타일의 갯수로 해볼까? -> 8!
- $h2(n)$ = **total Manhattan distance** Total 맨해튼 거리? -> 14
- sum(각각의 타일이 (최소로) 몇 번의 움직임에 제자리를 찾아 갈 수 있을까?)
- $h3(n)$ $h4(n)$ h는 다양하게 정할 수 있다!

Admissible Heuristic and Dominance 지배력

If $h2(n) > h1(n)$ for all n, then h2 dominates h1 and is better for search!
 모든 n에 대해 **$h2(n) > h1(n)$** 이라면, **h2**가 더 좋은 휴리스틱이다! (지배적이다?)
 지배력($h2 > h1$) : **예측값이 실제에 가까울수록(클수록) 쓸데없는 노드를 더 많이 쳐내서 빠릅니다.**

왜 그럴까? IDS로 하면 : 50,000,000,000개의 노드 expansion이 필요
 $A^*(h1)$: 39,135 노드
 $A^*(h2)$: 1,641 노드
 >> $A^*(h2)$ 를 했을 때 훨씬 더 빨리 수렴한다!

어떻게 하면 현재 나에게 주어진, 내가 현재 만난 문제에 최적인, 더 나은 h-func를 만들 수 있을까?
 => 입증은 실제 실험적 증거(실제적 성능, 속도, 수행시간) + 수학적 증거(proofs)를 바탕으로 해야 함.

AI에서 Optimization Problems란?

AI에서 어떻게 문제를 handle하

Fomulate : 문제를 Graph/Tree로 표현(by CS/AI 전공자)

문제를 Graph/Tree로 모델링하면 **상태공간**이 만들어짐. -> search -> solutions

탐색 알고리즘 : 최적화 문제의 suboptimal solution(optimal에 근접한 충분히 괜찮은 값)

알고리즘 시간 : BF(완전 탐색), B&B(분기한정법;pruning) -> Knapsack 문제

NP-Hard 문제, 최적화 문제 => Approximation Solutions 근사 해

Q 질문) max(goal-fn) Rational AI 정의.

수학적 수식으로 정의된 AI 문제는 어떻게 다른가?

Algebraic calc(미분) : max something, min something

함수 최적화 문제 : function/mathematical optimization

Jacobian, Iterative, Lagrange, Newton, Gradient.... 연속함수의 최적화 기법

조합 최적화 문제 : 루마니아 길찾기, Knapsack, TSP(외판원), 8-Puzzle

best-fit approach가 아님.

이산적, 조합적, discrete domain(combinations)으로 풀어야 함. 수학으로 풀 수 있는 것과 결이 다른 solution : 정답의 형태가 sequence(순서)이거나 combination(조합)이다!

이런 solution들은 미분 가능할까? -> 기본적으로는 미분 가능하지 않음.

미분이라는 것은 문제의 방향성을 추적.

=> AI algorithmic 접근 : search, heuristics, approximation, alternating opt.

3주차 Heuristics Design and Real-life Problems

A* 알고리즘 : $f = g + h$

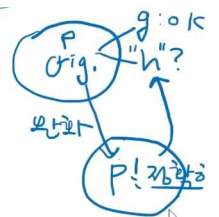
Q) 새로운 문제를 만나면 어떻게 h 함수를 만들 것인가?

Relaxation == Simplification

A) **Admissible heuristics**(h-func) can be derived from the "exact" solution cost of a "relaxed/simplified" version of the original problem

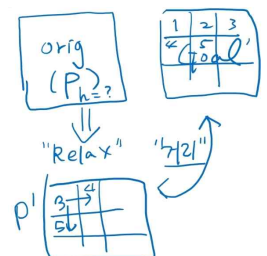
원래 문제를 조금 더 **relax하게 만들면**(제약 조건을 몇 개 없애면) 그 만만해진 문제에서 구한 "exact"한 솔루션을 원래 문제의 **Admissible heuristic(h-func)**으로 사용함.

$h_1(n)$ = **number of misplaced tiles** 잘못 놓여진 타일의 갯수로 해볼까? -> 8!
 $h_2(n)$ = **total Manhattan distance** Total 맨해튼 거리? 절댓값 거리 -> 14



1. 8-puzzle의 규칙을 **relax**해서 타일이 **어디든지** 움직일 수 있게 하면, $h_1(n)$ 이 최단 거리 solution을 제공함.
relax한 것(P')으로 찾은 "exact" 최단거리를 원본 원래 알고리즘인 P에서 사용.

2. 8-puzzle의 규칙을 relax해서 tile이 어떤 **인접한(adjacent) cell**로 움직일 수 있도록(옆에 누가 있어도 무시하고 움직이도록 허용) 하면,
 $h_2(n)$ 이 **최단 거리 solution**을 제공함.



-> optimal solution cost of a relaxed problem is **no greater than** the optimal solution of the real/original problem! relax된 문제의 optimal solution은 실제(원래) 문제의 optimal solution보다 **크지 않다**.

relaxed 문제의 최적 비용 ≤ 원래 문제의 최적 비용

-> **Admissible Heuristic**이 된다!

h가 점점 h^* 에 가까워지면서 점점 tight해진다!

Real-life Problem 실생활 문제

Local Search 새로운 인공지능 탐색 알고리즘 학습 <- 이것이 3주차의 목표이다.

전통적/기계적 탐색 : BFS, DFS, IDS ...

정보를 활용하는 탐색 : Greedy, B&B, A* ...

발전된/실제 현장에서 사용되는 **Local Search**

TSP(Travelling Salesperson Problem 외판원 문제) : CS에서 가장 어려운 문제의 클래스.

NP-Hard 문제다(엄청 어려운 문제 종류)

Hamiltonian Cycle 문제(한붓그리기 가능?)의 **최소비용** 버전.

NP Theory : transformation에 의해 다른 문제 환원.

가장 복잡도가 높은 조합 최적화 문제.

Q) 왜 Real-life 문제는 풀기가 어려울까?

1. Search space 탐색 공간 : 조합의 수 (optimal) 기하급수적으로 폭발한다!

- Time Complexity 시간복잡도를 결정한다.

- e.g. TSP 문제(최단거리로 한붓그리기)가 20개의 도시를 가지고 있다. Search Space의 크기는?

20 도시면 20팩토리얼 => $O(n!)$

10000개 도시면?? ->

2. Constraint 제약조건

- TSP : 반드시 거쳐야 하는 도시 or 거치면 안 되는 도시

- **Soft Constraint**(지키면 좋은 것) vs **Hard Constraint**(반드시 지켜야 하는 것)

3. Evaluation function 평가 함수 ; 어떻게 평가를 할 것인가?

- TSP : 1-3-4-.... 도시 이동 경로의 거리. $f = \text{dist}(1,3) + \dots + \text{dist}(7,9)$ => 평가하기가 명확하다!

그런데 실제 문제에서는 평가하기가 어려운 문제들이 多

4. Environment : dynamic 동적인 환경

- environment noise, dynamic factors 동적인 요인 ...

- TSP : 태풍, 출퇴근시간

Model의 중요성

Real-life 문제, AI 문제의 모델은 **최적화 형태**를 가진다.

Optimization problem : $\text{Max } f(x,y)$: "doing the right thing"

특히, 조합최적화 combinational optimization problem의 성격을 가짐.

combinational 문제는 이산 수학(discrete math)을 base로 함 답이 딱딱 떨어진다.(0/1)

대학원에서는 performance measure/evaluation이 여러 개인 경우를 해결하게 됨.

-> multi-object optimization problem

-> optimal에 근접한 Best 답을 제시함.

Modeling 모델링

어떤 형태로든 문제가 주어지면 답(Solution)을 구해야 함! 그것을 구할 때 model을 사용해야 함.

Problem -> Model -> Solution : solution to the "model" of the problem

Model은 다양하다! 따라서 Solution도 다양해진다

Problem은 자연어이다. 우리는 이 Problem을 코드로 Model을 만들어야 한다.

-> 한 가지 정답이 없다! 현실적으로 계산하기 어렵다.

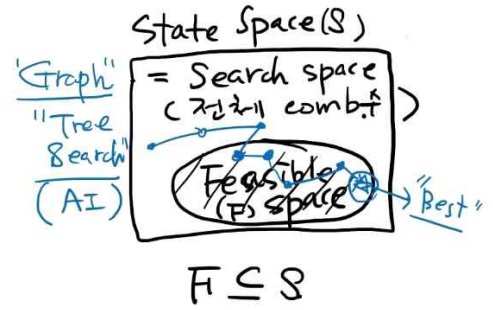
Feasible Solution

: 문제에서 주어진 제약조건을 만족하는 **solution**

State Space : 문제에서 가능한 모든 상태들의 집합

≈ Search Space : 탐색 공간. 알고리즘이 탐색하게 되는 전체 집합

Feasible Space : 제약을 만족하는 해의 공간.



Search Space $\supset$ Feasible Space $\supset$ Best Solution

Feasible Space(F) vs Search Space(S)

TSP문제에서는 $F == S$ 이다. 탐색하는 모든 후보가 곧 feasible solution이다.

Search Problem(AI)과 Optimization Problem(공학적)은 거의 같은 의미로 쓰인다. -> 둘 다 좋은 해를 찾는 방법 feasible solution 중에서 best solution를 찾는 것이 바로 optimization problem이다

TSP 문제에서 Problem과 Model

Problem : given **S(search space)** and **F(feasible space)**, find **x** such that **eval(x) ≤ eval(y)** for all **y** in **F**

탐색공간 S와 가능해공간 F가 주어졌을 때, feasible한 해들 중 **평가값이 가장 작은 해 x**를 찾아라

이동 거리가 제일 작은 것을 찾아야 하기 때문

★ **Model** : 아래 세 가지 요건을 만족시켜야 한다. **세 가지 외우기**

1. Representation : 후보 솔루션의 **형태를 표현함**.

해를 어떻게 표현할까?

: Search space를 결정함.

여기서는 도시의 방문 순서가 representation이 됨.

1-3-2-...8-9-3

-> n!개의 search space가 나오겠구나.

2. Objective function 목적함수 ★**핵심**★

무엇을 최적화할까?

: 수학적 표현. mathematical expression. 누구든지 그 표현을 보고 오해하지 않음.

: **min sum of distance(x,y)** x-y거리(도시간의 거리) 합 **의 최솟값**

3. Evaluation function 평가함수 ≈ scoring function

각 해를 어떻게 **점수화**할까?

: map each tour to its corresponding distance

AI에서의 탐색 알고리즘

Exhaustive search : "**partially**" constructed solutions **부분적으로** 만들어진 해를 가지고 평가하는 알고리즘

Local search : "**complete**" solution 완성된 해면 평가하는 알고리즘

Exhaustive search ≈ enumerative(나열식) search

: Classical DFS, BFS, backtracking, B&B, and A* algorithm...

candidates(solutions)를 탐색을 해 나가고 불필요한 것들은 pruning을 하면서 탐색

장점 :

1. simple
2. systematic : 기계적임. 모든 solution을 기계적으로 만들기 때문
3. pruning을 할 수 있음. 탐색 부담이 줄어듦. 성능이 올라감

단점 :

1. not practical 실용적이지 않음. (n>1000이면...) 천만개 도시가 있으면 조합 계산이 엄청남
2. 어떤 permutation은 feasible이 **아닐** 수도 있음! 실제 solution이 아니다.
3. pruning을 개선하려고 노력하고 있음...

Greedy Algorithm

: 여러 단계에 걸쳐 complete solution를 하나씩 구성해 나가는 방식
local optima choice(LOC)

1. 아주 **simple** : prototype 개발이나 구현이 쉬움
2. 항상 optimal solution을 제공하는 것이 아님! -> 그래서 증명을 해서 사용해야함.

TSP에서 Greedy Algorithm

가장 직관적이고 simple한 알고리즘은 "가장 가까운 이웃"을 찾는 heuristic이다.

Random city에서 시작

- > 가장 가까운 unvisited 도시를 찾음
- > 계속 진행
- > 첫 번째 도시로 돌아옴

Dynamic Programming 등등으로도 할 수 있음.

reminder : 0/1 Knapsack Problem

물건 집합 S 에서 일부 물건들의 집합 A 를 선택하되,
총 무게가 배낭 용량 W 를 넘지 않으면서
총 가치의 합이 최대가 되도록 하라.

Objective Function : $\max \sum_{i \in A} p_i$ 제약조건 : $\sum_{i \in A} w_i \leq W$

Evaluation Function : 또는 $\sum_A P_i$

Local Search

많은 optimization 문제에서 경로는 그다지 중요하지 않다. -> **goal state** 자체가 **solution**이다
로컬 탐색은 어떤 **특정 solution** 주변의 국소적인 이웃 영역에만 집중하는 탐색 방법이다.

1. state space == search space == "complete" solution
-> **추측(guessing)**을 해서 **초기 complete solution**을 찾는다.
e.g. 0/1 KS에서는 랜덤한 0/1 비트표현을 초기 solution으로 쓴다 : 1011 등
e.g. TSP에서는 랜덤한 순열조합을 쓴다. 1-3-2-...-8-9
2. constraints를 만족시키는 optimal solution을 찾는다.
3. 하나의 "**current**" solution을 유지하면서 점차 개선해 나간다. -> "iterative improvement search"
하나를 가지고 neighbor과 계속 비교하면서 optimal하면 바꾸는 식으로 수정해 나감. **Change한다!**
4. 장점 : (어떤 답이든) **반드시 solution**을 리턴한다. 반복적 개선을 통해 **향상된 답**을 도출할 수 있음
중간에 끊기더라도 반드시 답을 리턴.
HC(Hill Climbing), SA(Simulated Annealing), GA(Genetic Algorithm)

-> 어떻게 현재 solution을 더 좋은 solution으로 바꿀 것인가? -? **Transformation**을 어떻게 정의하는지가 중요!

3-6

1. search space에서 하나의 solution을 고르고, 그것을 current solution이라고 한다. 좋고 나쁨을 평가한다.
2. 현재 solution에 transformation을 적용하여 새로운 solution으로 하고 좋고 나쁨을 평가한다.
3. 새로운 solution이 더 좋으면 그것을 current solution으로 바꾸고, 그렇지 않으면 새로운 solution을 버린다.
4. 주어진 transformation 중 더 이상 좋은 solution이 안 나올때까지 2-3번을 반복한다.

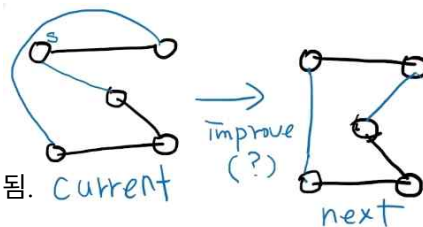
TSP에서 Transformation의 type

간단한 방법 : complete tour에서 아무거나 하나에서 시작해서, 두개를 바꾸는 교환(pair-wise exchange)를 수행
가장 단순한 방법은 **2-opt**(2-교환)이라고 부르는 방식이다.

Neighborhood는 서로 인접하지 않은 두개의 edge를 바꾸어서 도달할 수 있는 모든 순회 경로의 집합

-> 현재 해에서 조금 바뀌서 만들 수 있는 후보 solution의 모음

파란색으로 된 2개를 바꾸었다. ----->



반드시 2-opt를 해야만 하는 것은 아니다 3-opt, 4-opt를 해도 됨.

계속 evaluation을 해서 좋은 방향으로 나가야 함.

the **BEST local transformation**을 하기 위해서 노력해야 함.

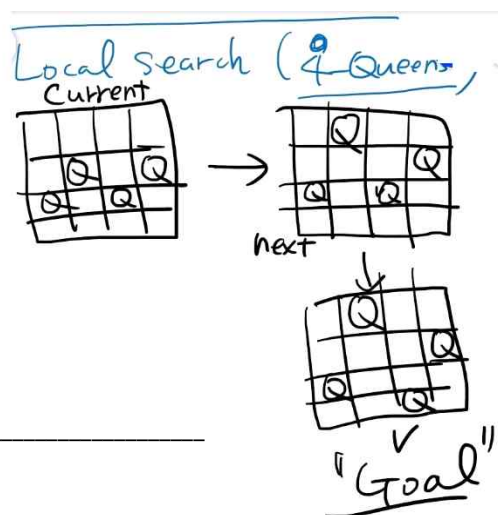
4-Queen 문제를 Local Search로 풀어보기

4x4 체스판에 퀸 4개를 서로 공격하지 않게 놓는 문제

랜덤으로 놓은 current에서는 서로 공격하는 불완전한 배치임.

-> 하나를 바꾼 next state임. **neighbor** 중 하나로 이동함

-> 더 좋은 상태가 되어서 결국 Goal이 됨.



이 문제를 DFS, BFS 등 전통적인 방법으로 풀면 매우 복잡함.

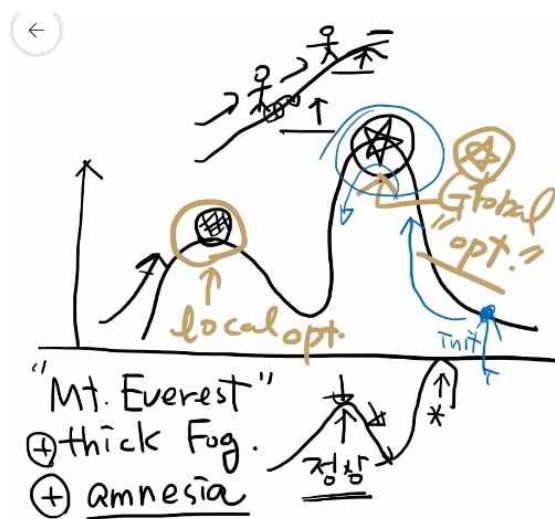
Hill-Climbing Algorithm (HC, aka gradient descent)

"반복적 개선!"

```
current <-- guess
loop do
  neighbor <-- transform(current)
  if (eval(neighbor) 비교 연산 eval(current))
    current <-- neighbor
end
```

local opt. - 지금 주변만 보면 제일 좋아 보이는 지점
but, global하게는 optimal이 아닐 수 있음

global opt. - 전체 탐색 공간에서 진짜 가장 좋은 해



hill climbing은 local optimal에 빠지기 쉽다.

하지만 결국 local optimal solution을 받아들여야 할 수 있다. 적당히 더 나은 선택

Q. How to avoid local optimal?

-> (simple) 지리산을 여러 번 여러 출발점에서 등반.

-> **random-restart** 을 하면 된다! -> **Random-restart Hill Climbing**

Q. Advanced algorithms? 더 발전된 알고리즘은 뭐가 있을까?

-> SA(Simulated Annealing), GA(Genetic Algorithm), ...

Problem, Model, Solution

Problem -> Model -> Solution

Model에서 **objective function**이 제일 중요!

e.g. 컴퓨터 네트워크 : 모델링을 할 때 prob/stat.(확률/통계) 기반하여 문제를 modeling함

e.g. 경제분야 : 선형대수를 사용해 경제 문제를 modeling함

모델링

중학교 수학: "코로나 바이러스 확산양상을 모델링하시오!"

수식, 그래프, 함수 형태로 나타내는 게 모델링. 어떻게 모델링 할지는 잘 생각해 봐야 함.

현실 현상을 보고

중요한 요소를 골라내고

어떤 관점으로 해석할지 정하고

그걸 수학적/계산적 구조로 바꾸는 것

인공지능 분야에서는?

AI 연구에서 정말 높은 평가를 받으려면 단순히 알고리즘 돌리는 게 아니라

문제를 어떻게 정의하고 objective function를 어떻게 세우느냐가 중요

4주차

일부 알고리즘은 global optima를 찾아내지만, 비용이 많이 든다 (**expensive**)

현실 문제들 중에는 **NP-hard**인 경우가 많고, 이런 문제는 정확한 optimal solution를 빠르게 구하는 **P-time 알고리즘이 없다**고 보는 경우가 많다.

-> 그래서 Local Search를 사용한다.

구현하기 쉽고, complete solution에서 시작한다.

but, 좋지않은 local optima에 빠질 수 있다.

"Local Optimal"

이것이 과연 나쁠까?

-> 실제로 중요한 것은 무조건 global optimum을 찾는 것이 아니라, 현실적으로 **충분히 좋은 local best optimal**에 빨리 수렴하는 것

Q. 데이터의 폭발적 증가

DB 분야에서는 데이터가 엄청나게 늘어나고 있고, 그런 상황에서 쓸모없는 데이터, 덜 중요한 데이터, **불필요한 부분을 어떻게 ML/AI로 걸러낼 것인가**가 중요하다.

-> 'n'이 폭발적으로 증가하면서, 'greedy'나 'local search'가 practical한 solution이 되고 있다.

대부분의 문제는 **NP-hard**이다. 이것을 해결하려면..

1. random restart (여러 곳에서 등산 시도)
2. probabilistic approach 확률적 접근
3. increase the size of neighbors (여러 명 시도)

one possibility 시도해 볼 것 : **iterative hill-climbing** 여러번 **hill-climbing**을 해 보기

-> 2 가지의 다른 **Approach**도 있음.

1. **Simulated Annealing**
2. **Tabu Search**

Simulate Annealing

-> moving from one point of the search space to another

더 나쁜 이웃 **solution**도 초기에 일정 확률로 받아들인다

그래서 local optimum에서 탈출할 수 있음

시간이 지나며 T를 줄여서 후반에는 ordinary hill climbing처럼 수렴

Tabu Search

based on memory, which forces the algorithm to explore new points of the space.

메모리(memory)에 기반하며, 알고리즘이 탐색 공간의 새로운 지점들을 탐색하도록 강제한다.

방금 갔던 곳, 최근에 했던 선택, 이미 시도한 방향 등을 기억 -> 계속 같은 곳만 맴도는 것 방지.

강제로 새로운 solution을 찾게 함.

Key Idea : newly generated neighbor **does NOT have to be a Better solution.**

새로 생긴 솔루션이 반드시 더 좋을 필요는 없다.

무조건 "좋아진 경우만 이동"하는 게 아니다.

-> accept the new solution with **P(확률): stochastic(확률적인) HC.**

Stochastic Hill Climbing

t <- 0

select a current Vc (random)

evaluate Vc

repeat

Create the new Vn from Vc

select "Vn" with "Probability"

t <- t+1

until t <- MAX

$$P = \frac{1}{1 + e^{\frac{\text{eval}(V_c) - \text{eval}(V_n)}{T}}}$$

P: 새 해를 받아들일 확률 (0~1)

P가 1이면 새 해가 더 좋다 **0이면** 새 해가 별로다

Vc : current solution, 현재 해

Vn : new solution, 새 해

T: temperature 같은 조절 파라미터(**상수**) 전체 수식에 dominance를 가지고 있다!

e.g. Maximization Problem을 가정하자. (0/1 Knapsack)

eval(Vc)=107, eval(Vn)=120 라 하자. ----->

T값을 변화시켜 보자

$$\text{val}(\quad) - \text{eval}(V_n) = 107 - 120 = -13$$

$$P = \frac{1}{1 + e^{-13/T}}$$

경우 1) T = 무한인 경우

$$T = \infty : P = \frac{1}{1 + e^{\frac{-13}{\infty}}} = \frac{1}{1 + e^0} = \frac{1}{1 + 1} = \frac{1}{2} = 0.5$$

Random Search이다

경우 2) T = 1 인 경우

$$T = 1 : P = \frac{1}{1 + e^{\frac{-13}{1}}} = \frac{1}{1 + e^{-13}} = \frac{1}{1 + 0} = 1.0$$

Ordinary HC 일반적인 HC 문제이다!

e^{-13} 은 매우 작은 수라 0 취급

>> The greater the value of T, the smaller the impact of "Vc - Vn"

>> T 값이 클수록, $\text{eval}(V_c) - \text{eval}(V_n)$ 의 영향이 작아진다. 결과가 둔감해진다. inf => 0.5

>> T 값이 작을수록, $\text{eval}(V_c) - \text{eval}(V_n)$ 의 영향이 더 커진다. 결과가 민감해진다. 1 => 1

e.g. Maximization Problem을 가정하자.

그런데 이번엔 T를 10으로 고정! eval 값을 변화시켜보자

T = 10으로 고정시킨다

Vc를 107이라 하고 Vn 을 변화시키면

Eval(V_n)	Difference ($107 - V_n$)	$P = \frac{1}{1+e^{(107-V_n)/10}}$	값
80	27	$\frac{1}{1+e^{27/10}}$	0.06
107	0	$\frac{1}{1+e^0}$	0.50
150	-43	$\frac{1}{1+e^{-43/10}}$	0.99

Vn의 값이 커질수록 Difference가 작아지므로 P 값이 커진다!

4-3

Simulated Annealing

Stochastic Hill Climber와 Simulated Annealing 의 차이는 뭘까?

Stochastic Hill Climber에서 T가 상수였는데 이것을 변수화 시키고자 한다! -> 실행 중에 T 값을 조정하자!

Annealing : 화학에서 '담금질'

SA는 T(temperauture)를 runtime에 변경시키는 것이 주된 아이디어이다!

SA는 T를 높은 값부터 시작한다.

(Bad Move도 받아들임. Random Search처럼 작동)

-> 점진적으로 T 값을 낮춰나감.

-> 탐색이 끝나갈 무렵 T가 작아짐.

(거의 Ordinary Hill-Climbing처럼 작동)

Procedure of SA

begin

t <- 0

T <- initial value //아주 큰 값 9999... or inf

select Vc (guess) //Vc를 추측

evaluate Vc //Vc를 평가

Repeat

Repeat

select Vn using Vc

if Vn is better than Vc, then Vc <- Vn // Vn이 더 나으면 대체!

else if (rand() < P), then Vc <- Vn // 랜덤으로 뽑은 숫자가 P 값보다 좋다면,

// Vn으로 대체하는 Bad move도 허용한다. 나쁜 길을 가보자

until (termination-cond.)

T <- g(T, t) // 감소

t <- t + 1

Until (termination-condition) //e.g. t = tmax ?

end

초록색 글씨 파트는 수정할 수 있다!

Questions

1. How to determine the initial temperature T

초기 T를 어떻게 결정할 것인가?

2. How to determine the cooling ratio g(T,t)

cooling하는 감소함수 g(T,t)를 어떻게 정할 것인가?

(추가) 3. 어떻게 Termination-condition을 어떻게 정할 것인가?

Tabu Search

Tabu search uses a memory("state") to force the search to explore new areas of the search space.
 Tabu search는 메모리(상태 정보)를 사용해서, 탐색이 탐색 공간의 새로운 영역을 탐색하도록 강제한다.

최근에 살펴본 몇몇 solution들을 기억해 두고, 그 해들은 **tabu solution**이 된다.
 -> 다음 solution를 고를 때 그 solution들은 피한다.

Stochastic Hill Climbing, Simulated Annealing : Probabilistic(확률적) 탐색 알고리즘이다.

-> 실행(run)할 때마다 결과가 달라질 수 있다. 랜덤 값 등등 때문

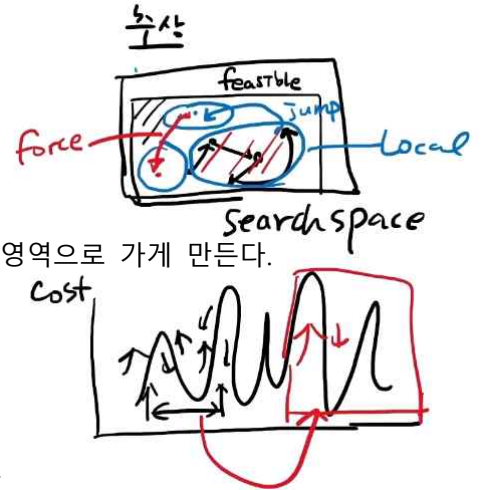
Tabu search : Deterministic(결정적) 탐색

-> 실행할 때마다 같은 결과가 나온다!

The **BEST non-tabu solution** is accepted for the next iteration.
 tabu가 아닌 것 중에 가장 좋은 solution을 고름

>>Tabu Search는 최근 본 해를 금지해서, local 근처만 맴돌지 않고 다른 영역으로 가게 만든다.

Tabu search 알고리즘은 파생형이 많고, 쉽게 구현 가능하다.



Local search with Tabu List

1. **Recency-based memory** : 최근 몇 단계 동안의 행동 action을 기억한다.
2. **Frequency-based memory** : 무엇을 자주 했는지 기억한다.

빈도!

e.g. Tabu Search for 8-city TSP

특정 해에서 두 도시의 위치를 SWAP하는 이동을 생각해보자.

Q. 28가지 SWAP하는 방법이 있다. 다음에 뭘 할까?

Vc : 7-3-5-6-1-2-4-8

Vn : 7-5-3-6-1-2-4-8

M

$M[2,6] = 5$

city	1	2	3	4	5	6	7	8
1	0	0	0	1	0	0	0	0
2		0	0	0	0	5	0	0
3			0	0	0	0	4	0
4				0	3	0	0	0
5					0	0	0	2
6						0	0	0
7							0	0
8								0

Recency-based memory -----

$M[2,6] == 5$ 라는 것은 제일 최근에 2-6 SWAP을 했다는 뜻

(previous) 7-3-5-2-1-6-4-8 ==> (current) 7-3-5-6-1-2-4-8 이다.

-> 2-6 SWAP은 앞으로 5번동안 SWAP 금지다! **tabu!** matrix에서 0인 곳(unvisited)을 탐색!

Frequency-based memory

Recency-based보다 matrix에 정보가 더 많다.

$M[7,8]=6$ 이라는 것은 7-8 SWAP이 제일 자주 일어났다는 뜻

3-8 같은 쌍은 아직 한번도 SWAP되지 않음

-> 이런 조합이 **preferred candidates**

그렇다면, local optima문제를 escape하기 위해서는 어떤 정보들 넣는 것이 좋을까? -> 많은 방법이 있다!

city	1	2	3	4	5	6	7	8
1	0	0	2	3	3	0	1	1
2		0	2	1	3	1	1	0
3			0	2	3	3	4	0
4				0	1	1	2	1
5					0	4	2	1
6						0	3	1
7							0	6
8								0

'Frequency-based Tabu' = 6

$M[7,8] = 6$

★ The more sophisticated the method, the more you have to use your judgement as to how it should be utilized: complex/complicated algorithm -> simple and straight-forward algorithm.

알고리즘이 정교해질수록 성능 잠재력은 커지지만, 파라미터 설정과 활용 방식에 대한 사용자의 판단도 더 많이 필요하다. 처음에 만든 알고리즘은 복잡할 수 있겠지만 결국에는 심플하고 직관적으로 일반화하는 것이 좋다!

5-1

4주차 정리

Hill-Climbing은 **deterministic** ruled을 사용한다 -> **같은 상황이면 항상 같은 선택**을 한다.

neighbor solution이 있을 때, 더 좋은 해가 있으면 무조건 가고 아니면 안감. -> 선택이 항상 일정.

SA는 probabilistic이다. 같은 상황이어도 확률에 따라 선택이 달라질 수 있음.

local optimum을 탈출할 수 있음. 확률적으로 weak(bad) solution을 받아들임.

시간에 따라서 T(Temperautue)을 낮춰가면서 탐험->...-> 정착을 하자.

지리산을 등반하는데, 한 사람이 한 출발점에서 시작하는 것이 아니라 여러 출발점에서 출발해보자!

- 하나의 해만 유지하는 대신, **여러 개의 해를 동시에 유지한다면** 무슨 일이 일어날까

얼핏 보면, 좋아 보이는 것 같지만 실제로는 새롭을 게 아무것도 없다...

-> 병렬 컴퓨팅과 동일하게 작동함. parallel algorithm. CPUs, GPU...

-> 반복적인(iterative) hill-climbing algorithm.

Genetic Algorithm

e.g. 세계적인 농구 선수를 육성해보자.

방법 1) 선수 한명을 계속해서 계속 improve

방법 2) 여러명의 선수를 선발해서 improve하기

방법 3) 우수한 남자 농구선수 + 여자 농구선수 -> 이 방법이 좋지 않을까?

생물학에서 아이디어를 얻어 옴!

진화에 의한 발전 -> 컴퓨팅

1. **Evolution 경쟁**과 **Selection 적자생존**의 "**Evolutionary Process 진화 과정**"을 이용하고, candidates solution들이 미래 세대에서 살아남을 자리를 두고 경쟁하게 한다.

2. 새로운 solution를 찾기 위해 **random variation(돌연변이)**을 준다. -> 자연적인 진화

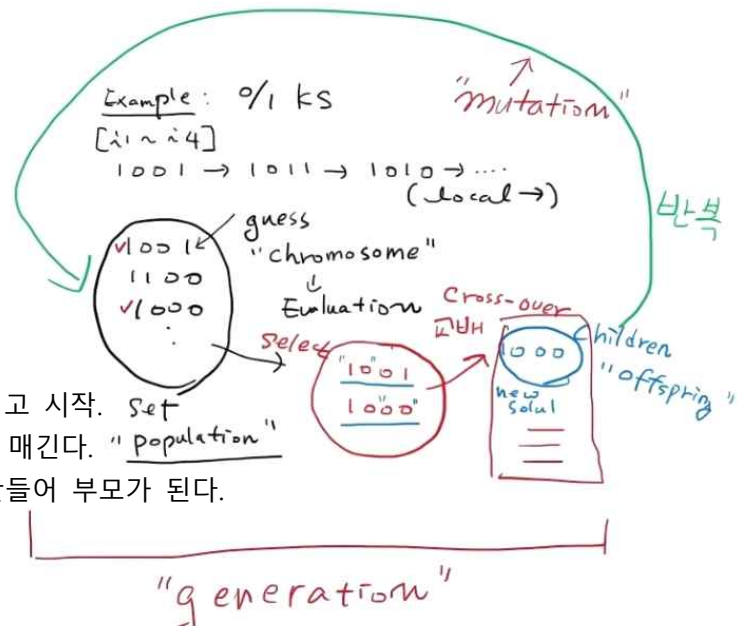
이렇게 함으로서 local optima를 탈출함.

Genetic Algorithm의 순서

1. start with a **set of** initial solution.
2. Question : 어떻게 offspring 자손을 만들 것인가?

기본 아이디어

1. initial solution의 **집합**으로 시작한다.
답 하나가 아니라 여러 개의 후보 답을 가지고 시작. **Set**
2. evaluation function(fitness)를 사용해 점수를 매긴다. "**population**"
3. 더 좋은 solution이 다음 자손(offspring)을 만들어 부모가 된다.



Offspring 자손을 어떻게 만들까?

1. We don't have to rely on the neighbors of each solution.
각 해의 이웃(neighbor)만 보고 조금씩 바꾸는 방식에만 의존할 필요가 없다.
2. We can use more than one parent solution to generate a new candidate solution.
새로운 후보해를 만들 때 부모 해를 하나만 쓸 필요가 없다. -> 부모 여러개 사용
3. one way is by taking parts of two parents and putting them together to form a new offspring.
두 부모의 일부를 가져와 합쳐서 새 자식 해를 만들 수 있다. **crossover(교배)!**
4. with each generation, the individuals(solutions) compete ... for "inclusion" in the next iteration
각각의 generation동안 각각의 individual(solution)은 경쟁을 한다. 세대를 반복하면서 경쟁을 함.
5. After a series of generations, we finally observe a succession of improvements and a convergence.
여러 세대를 거치고 나면 개선(improvements)이 이어지고, 결국 어느 정도 안정된 좋은 해 쪽으로 수렴(convergence)하게 된다.

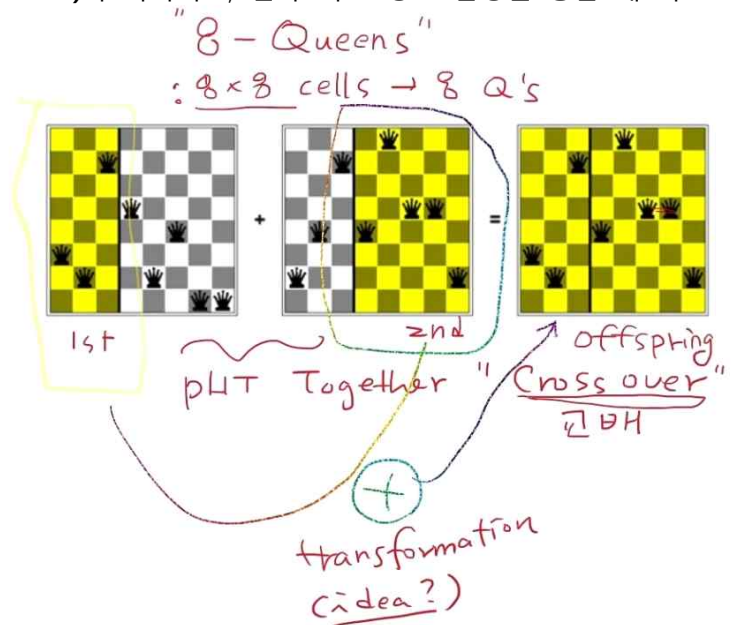
e.g. 8-Queens problem

8개의 퀸이 서로 공격하지 않게 하는 문제

부모 solution 2개가 있음.

한 부모의 일부 + 다른 부모의 일부

-> crossover를 해서 새로운 해를 만들



-> "WHY should we labor to solve a problem by calculating difficult mathematical expressions or developing a complex computer algorithm to optimize the results?"

어떤 문제는 수학적으로 정확하게 optimal solution를 구하려고 하면 너무 어렵다.

그럴 때 유전 알고리즘처럼 유연한 휴리스틱(heuristic) 방법을 쓰면 실용적으로 좋은 답을 찾을 수 있다!

부르는 명칭이 다양하다.

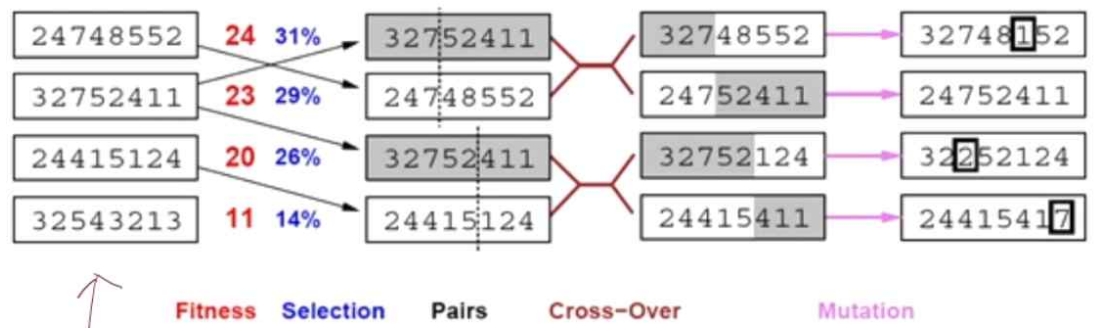
Genetic Alogrithm, Genetic Programming, Evolutionary Algorithm...

"Swiss Army Knife algorithm in CS" 만능 칼이다!

begin

```
t = 0 //타임스탬프
initialize P(t) // population-set 시작 후보들 준비
evaluate P(t) //각 후보들의 fitness 적합도를 계산함.
while (not termination-condition) do //종료 조건이 만족되지 않는 동안..
begin
t = t + 1 // 세대 증가
select P(t) from P(t-1) //좋은 부모를 선택
alter P(t) //P(t)를 변형시킴
evaluation P(t) // P(t)를 평가
```

end
end



만들기 → 평가하기 → 고르기 → 바꾸기 → 다시 평가하기

The idea is quite simple: a population (set) of solutions is evolved over iterations(steps) of "**variation**" and "**selection**".

해들의 집합(population)은 **variation**과 **selection**의 반복을 통해 진화한다.

1. Variation (변형)

mechanism for discovering new solutions → **crossover and mutation**

새로운 solution를 발견하는 장치 **crossover(교배) + mutation(돌연변이)**

2. Selection (선택)

determination of which solutions to maintain as a basis for the next(further) exploration.

다음 탐색의 기반으로 어떤 **solution**를 유지할지 결정하는 과정

TSP를 GA로 풀 때의 4가지 핵심 이슈

- 1) **Representation (solution)** 해를 어떻게 표현할까?
- 2) **Variation operators** crossover, mutation을 어떻게 할까?
- 3) **Evaluation function** 어떤 해가 좋은 해인지 어떻게 점수화할까?
- 4) **Selection** 어떤 해를 다음 세대에 부모로 할까?

1. Representation (solution)

a mapping from the state space of possible solutions to a state space of encoded solutions within a data structure.

원래 문제의 해를 컴퓨터 안에서 다루기 쉬운 형태로 바꾸는 것

- 1) A simple way <**Permutation**> is to make an ordered list of the cities to visit.

순열 표현 : 가장 단순한 방법은 방문할 도시 순서를 그대로 적는 것이야.

e.g. <1 3 5 10 4 2 8 9 7 6> 직관적으로 1->3->5.. 이런 순서로 간 것

일기는 쉬우나, 자식을 만들었을 때, TSP의 솔루션 형태를 만족시키지 못할 수 있다. 중복 可

- 2) <**Adjacency**> representation: city j is listed in the position i if and only if the tour leads from city i to city j.

인접 표현 : 이건 "방문 순서"를 직접 적는 게 아니라, i번 도시에서 다음에 어디로 가는지 적는 방식

e.g. The vector <2 4 8 3 9 7 1 5 6> represents the tour 1-2-4-3-8-5-9-6-7.

참고! ->

- 1에서 2로 감
- 2에서 4로 감
- 4에서 3으로 감
- 3에서 8로 감
- 8에서 5로 감
- 5에서 9로 감
- 9에서 6으로 감
- 6에서 7로 감
- 마지막엔 다시 시작점으로 이어짐

Q) <adjacency> 이걸 도대체 왜쓸까...?

A) It allows us to look for templates that are associated with "good" solutions.

좋은 해에서 자주 나타나는 연결 패턴을 찾기 좋다!

3) <Ordinal representation> : the i-th element of the list is a number in the range from 1 to n-i+1.
현재 남아 있는 후보들 중 몇 번째를 고를까? 절대적인 도시 번호가 아니고 상대적인 순서를 적는 것.

e.g. Given a list (1 2 3 4 5 6 7 8 9), a tour 1-2-4-3-8-5-9-6-7. ==> ord. rep: <1 1 2 1 4 1 3 1 1>

처음에는 [1, 2, 3, 4, 5, 6, 7, 8, 9] 이걸 기준으로 보기

1) 첫 도시 1

남은 리스트 [1,2,3,4,5,6,7,8,9] 에서
1은 1번째
→ 1

1 제거 후:
[2,3,4,5,6,7,8,9]

2) 다음 도시 2

남은 리스트에서 2는 1번째
→ 1

2 제거 후:
[3,4,5,6,7,8,9]

3) 다음 도시 4

남은 리스트에서 4는 2번째
→ 2

4 제거 후:
[3,5,6,7,8,9]

4) 다음 도시 3

남은 리스트에서 3은 1번째
→ 1

3 제거 후:
[5,6,7,8,9]

5) 다음 도시 8

남은 리스트에서 8은 4번째
→ 4

8 제거 후:
[5,6,7,9]

6) 다음 도시 5

남은 리스트에서 5는 1번째
→ 1

5 제거 후:
[6,7,9]

7) 다음 도시 9

남은 리스트에서 9는 3번째
→ 3

9 제거 후:
[6,7]

8) 다음 도시 6

남은 리스트에서 6은 1번째
→ 1

6 제거 후:
[7]

9) 마지막 도시 7

남은 리스트에서 7은 1번째
→ 1

그래서 최종 결과가
<1 1 2 1 4 1 3 1 1>

순서 정보만 가지고 있다! -> 자식으로 생성되는 것이 **offspring feasible**하다!

-> A good representation leads to good search(variation) operation
representation을 잘해야 좋은 탐색이 된다!

5-5

1. Representation : permutation, adjacency, ordinal

the choice of how to represent a tour is related to the choice of what variation operators to use.
투어를 어떻게 표현하느냐는, 어떤 **variation operator**를 쓸지와 연결되어 있다.

2. Variation operators : crossover (PMX, OX), mutation

PMX : Partially Mapped Crossover 부분 매핑 교차

it builds a offspring by choosing a **subseq. of a tour from one parent** and preserving the order and position of as many cities as possible from the other parent.

PMX는 한 부모에서 어떤 **부분 구간(subsequence)**을 선택하고, 나머지 부분은 다른 부모의 정보도 최대한 살리면서 자식 해를 만든다.

e.g. assume <Path Permutation> representation
a tour 5-1-7-8-9 is represented as <5 1 7 8 9>

이 둘로부터 어떻게 offspring를 만들까?

P1 = <1 2 3 4 5 6 7 8 9>

P2 = <4 5 2 1 8 7 6 9 3> 이렇게 2개의 도시가 있다고 하면

A subsequence of tour is selected by choosing **two random cut points**, which serve as **boundaries** for the swapping operations.

두 개의 random cut points(무작위 절단점)를 골라서, 그 사이 구간을 교환/복사하는 기준으로 삼는다.

1. 부모 하나에서 **특정 구간**을 고정
2. 다른 부모를 참고해서 **나머지를 채움**
3. 이때 **중복을 피하려고 mapping(대응 관계)**을 사용
4. 최종적으로 유효한 순열을 만든다

$$P_1 = \langle \underline{1\ 2\ 3} \mid 4\ 5\ 6\ 7 \mid \underline{8\ 9} \rangle$$

$$P_2 = \langle \underline{4\ 5\ 2} \mid 1\ 8\ 7\ 6 \mid \underline{9\ 3} \rangle$$

$$O_1 = \langle \underline{4\ 2\ 3} \mid 1\ 8\ 7\ 6 \mid \underline{5\ 9} \rangle$$

$$O_2 = \langle \underline{1\ 8\ 2} \mid \overset{\text{from } P_1}{4\ 5\ 6\ 7} \mid \overset{\text{from } P_2}{9\ 3} \rangle$$

OX : Ordered Crossover 순서 교차

it builds offspring by choosing a subseq. of a tour from one parent and **preserving the "relative order"** of cities from the other parent.

OX는 한 부모의 부분 구간을 고른 뒤, 다른 부모의 도시들을 그 "**상대적 순서(relative order)**"를 유지하면서 채워 넣는 방식이다.

OX

$$P_1 = \langle \underline{1\ 2\ 3} \mid 4\ 5\ 6\ 7 \mid \underline{8\ 9} \rangle$$

$$P_2 = \langle \underline{4\ 5\ 2} \mid 1\ 8\ 7\ 6 \mid \underline{9\ 3} \rangle$$

from P2

from P1

$$O_1 = \langle \underline{2\ 1\ 8} \mid \underline{4\ 5\ 6\ 7} \mid \underline{9\ 3} \rangle$$

$$O_2 = \langle \underline{3\ 4\ 5} \mid \underline{1\ 8\ 7\ 6} \mid \underline{9\ 2} \rangle$$

seq. of cities in P2

"9 3 4 5 2 1 8 7 6"

"9 3 2 1 8"

mutation : use a probability that an arbitrary point in a solution is changed from its parent.

돌연변이 : 일정 확률(probability)로, 해의 어떤 한 부분(arbitrary point)을 부모와 다르게 바꾼다.

-> mutation을 섞을 확률을 잘 컨트롤 해야한다. 확률이 너무 높으면 Random이 되어버린다.

$$O_1 = \langle \underline{4\ 2\ 3} \mid 1\ 8\ 7\ 6 \rangle$$

"2-bit mut"

<< 멀쩡한걸 갑자기 바꿈

3. Evaluation function (in TSP): total length (easy)

evaluation = fitness = objective function.

TSP에서 평가 함수는 전체 경로 길이(total length)다.

4. Selection

어떤 해를 다음 세대에 남길까?

The selection acts on individuals in the current population. Based on each quality, it determines the next population.

selection은 현재 population 안의 개체들에 작용하고, 각 개체의 품질(quality)을 바탕으로 다음 population을 결정한다. -> 현재 세대 개체들 중 누가 살아남고, 누가 부모가 되고, 누가 더 많이 선택될지를 정하는 단계

복원추출 vs 비복원추출

- 1) some individuals are eliminated from consideration

비복원추출(다시 안 넣음. 한번만)

어떤 개체들은 아예 탈락하고, 남은 개체들도 최대 한 번만 기여.

- 2) Individuals are sampled with replacement

복원추출(다시 넣음)

좋은 개체는 여러 번 선택될 수도!

Deterministic vs Stochastic Selection

1) Deterministic selection

always eliminates the same individuals, given the same population. It is faster than stochastic selection. population이 주어지면 항상 같은 개체를 탈락시킴.

2) Stochastic selection

generates a prob. over the possible compositions of the next generation

확률적으로 다음 세대 분포를 만듦.

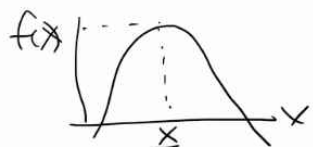
e.g. random tournaments : 항상 같게 할까? 아니면 랜덤하게 할까?

Genetic Algorithm 예제

Example: function 최대화 (max) with ONE var.

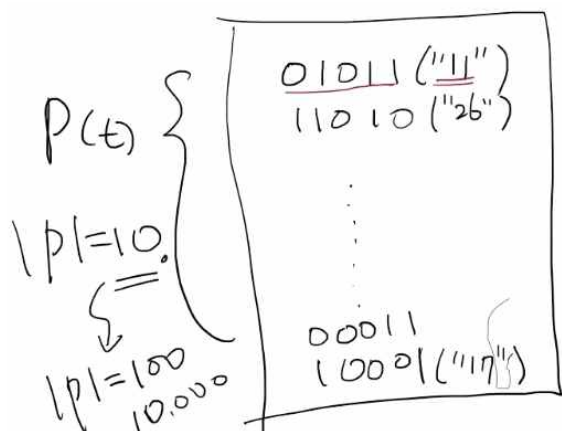
$f(x) = -\frac{x^2}{10} + 3x$. → ① 미분: math. opt. 기호 (by Goldberg's test)

→ 수치해석 but, 미분 불가? → large-scale opt: Heavy computation

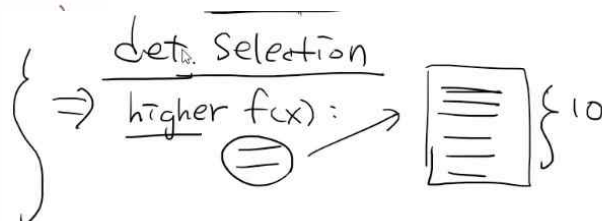


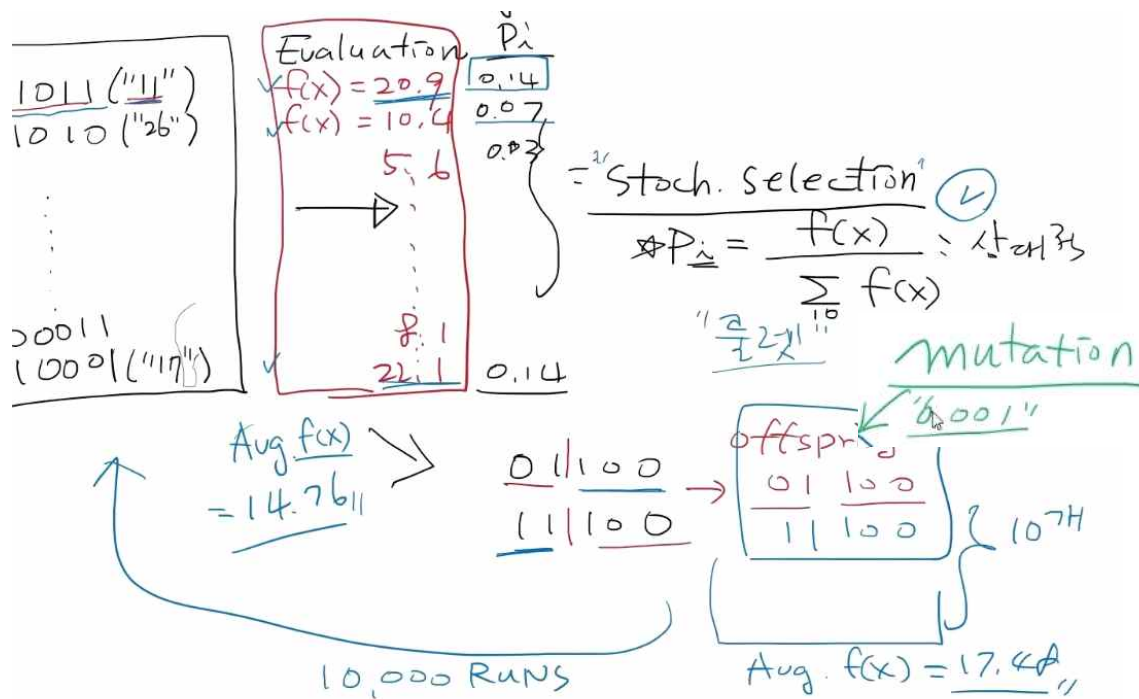
→ ② "Search" 해를 찾자 (subopt.)

⇒ Genetic Alg ① Repⁿ: $x = \text{integer} \rightarrow \text{crossover (?)}$
 ↳ "binary bit" "8" = 01001
 "9" = 01010
 "3" = 00011
 ↳ "5-bit binary"



Evaluation
 $f(x) = 20.9$
 $f(x) = 10.4$





Genetic Algorithm의 단점

1. 다른 알고리즘보다 매우 느림!
2. 파라미터가 매우 많음!

-> 시간이 많이 걸리고, 좋은 디자인을 위한 좋은 휴리스틱을 만드는 데 상당한 노력이 든다

5-7

GA 실습

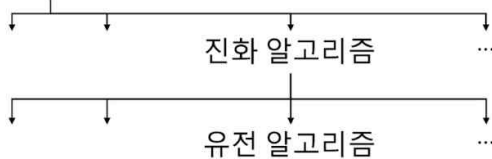
Metaheuristic

=> Meta + Heuristic

=> (넘어서) + Heuristic

=> 다양한 문제에 적용하기 위한 휴리스틱 기반 방법론

Metaheuristic



진화 알고리즘

1. 초기화
-> 후보군 생성
2. 평가
-> 생성된 후보군 평가
3. 선택
-> 재생산에 쓸 후보군 선택
4. 재생산
-> 선택된 후보군 바탕으로 재생산
5. 2번으로 복귀 또는 종료

유전 알고리즘

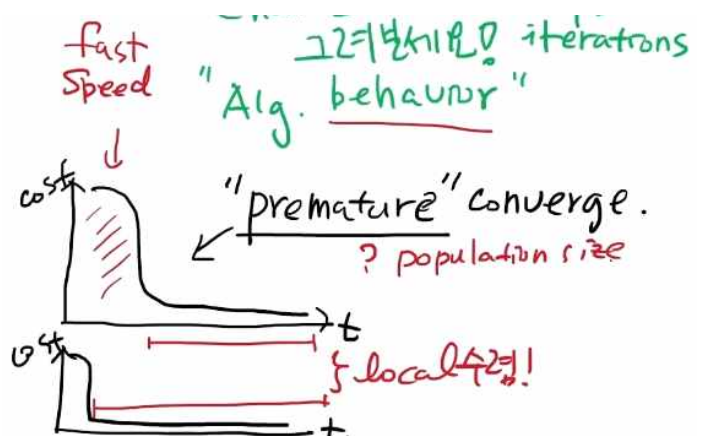
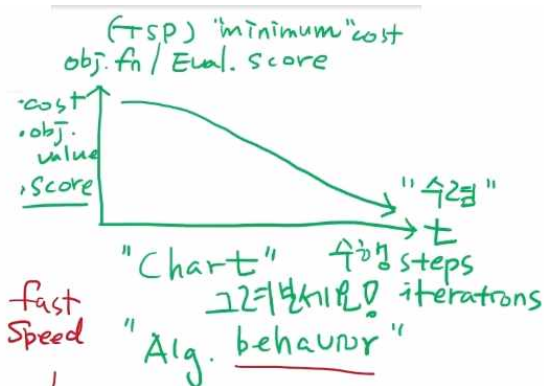
1. 초기화
-> 유전자 풀 (Pool) 생성
2. 평가
-> 생성된 유전자 평가
3. 선택
-> 재생산에 쓸 유전자 선택
4. 재생산
-> 유전자 교배 및 돌연변이
5. 2번으로 복귀 또는 종료

Premature Convergence 조기 수렴

(TSP) "minimum" cost - TSP에서는 cost가 최소화 되면 될수록 좋음

Premature Convergence : 아직 더 좋은 해가 있을 수 있는데, 알고리즘이 너무 일찍 한 해에 갇혀버리는 현상

GA라면 population size때문일까?



Inference 추론

개요 :

Knowledge-based agents
 Wumpus world : 예제 세계
 Logic in general : 명제
 Propositional logic : 명제 논리
 Inference rules and theorem proving : 추론 규칙, 정리 증명

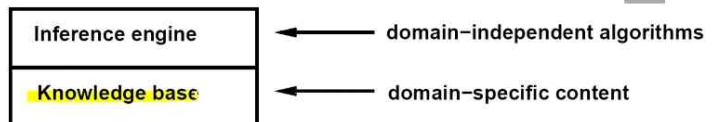
용어 :

intuitive reasoning vs logical reasoning 직관적인 추론 vs 논리적인 추론
 deduction or deductive reasoning 일반적인 규칙에서 구체적인 결론을 끌어내는 방식
 induction or inductive reasoning 여러 개의 구체적 사례를 보고 일반적인 규칙을 만드는 방식
 abduction or abductive reasoning 관찰된 결과를 가장 잘 설명하는 원인을 추정
 inference 추론

Knowledge Base

Inference engine : 도메인(분야) 독립적. 알고리즘
 Knowledge base : 도메인 특화 내용. 데이터베이스

domain-independent algorithms
 domain-specific content



KB는 형식 언어(formal language)로 표현된 문장들의 집합이다

TELL it what it needs to know 에이전트에게 알아야 할 것을 알려줘라

ASK itself what to do: answers should follow from the
 스스로 무엇을 해야 하는지 물을 수 있다. 답은 KB로부터 따라나와야 한다

Wumpus World

Start : 사냥꾼 시작

Wumpus : 괴물 위치

Stench : 악취. 괴물 주변엔 악취가 남

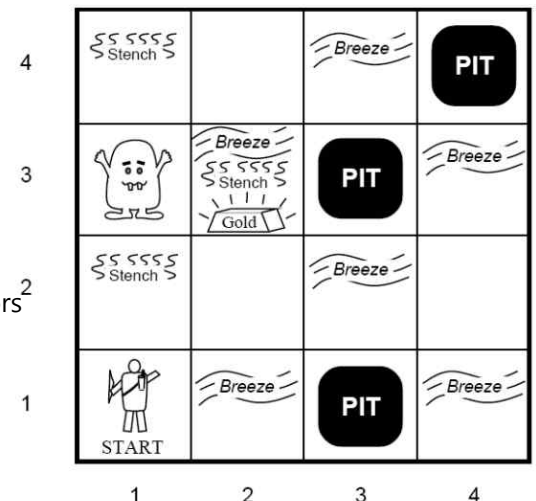
PIT : 함정

Breeze : 바람. PIT 주변엔 악취가 남

PEAS P: Performance measure E: Environment A: Actuators S: Sensors

Environment

wumpus 주변엔 악취가 남
 PIT 주변엔 바람이 불
 gold가 있는 칸은 glitter 반짝임
 wumpus는 죽일 수 있음. 화살은 하나다.(?)
 금을 grab하거나 release할 수 있음.



Performance Measure

gold 획득 +100 죽음 -100 한번 움직임 -1 화살쏘기 -10

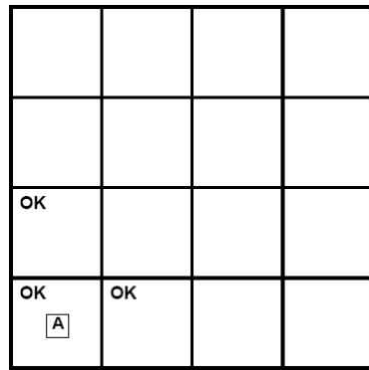
Actuators(행동)

앞으로 이동 잡기 놓기 화살쏘기

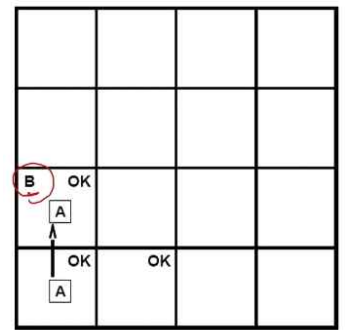
Sensors

냄새 반짝임 바람

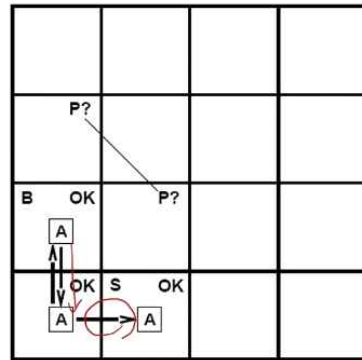
안전하다.
위 오른쪽 다 갈수 있음



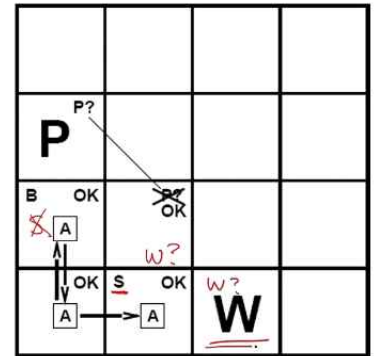
위에 바람!
-> 위, 오른쪽 가기
망설임



다시 올라와서 오른쪽으로
그런데 Smell이 남
옆에 Wampus가 있나?
오른쪽으로 가보자



아까 위로 갔을 때
Smell이 없었으므로
W는 (3,1)에 있겠군!
그리고 현재 Br가 없으니
P는 좌표(1,3)에 있겠군!



Logic = Syntax + Logic

Logic 논리

- : **"formal" language** for representing information such that conclusions can be drawn
- : 논리는 결론을 이끌어낼 수 있도록 정보를 표현하는 **"형식적" 언어**이다

Syntax

- : Syntax defines the sentences in the language
- : 구문은 그 언어에서 **어떤 문장이 가능한지** 정한다.

Semantics

- : Semantics define the meaning of sentences; define the truth of a sentence in a world
- : Semantics은 **문장의 의미**를 정한다. 또 어떤 세계(world)에서 그 문장이 **참인지**도 정한다

Entailment

어떤 것이 다른 것으로부터 따라 나온다는 뜻.

KB $\Rightarrow \alpha$ **KB로부터 α 가 따라나온다!**

지식베이스 KB가 문장 α 를 entail한다는 것은, 그리고 오직 그럴 때는,
KB가 참인 모든 세계에서 α 도 참이라는 뜻이다

e.g. "자이언츠가 이겼다"와 "레즈가 이겼다"를 포함하는 KB는 "자이언츠가 이겼거나 레즈가 이겼다"를 entail한다
e.g. $x+y=4$ entails $4=x+y$

entailment는 syntax들 사이의 관계이지만, 그 관계는 semantics에 근거한다

e.g. [1,1]에서 아무것도 감지하지 못한 뒤, 오른쪽으로 이동했고, [2,1]에서 바람(Breeze)을 느낀 상황

KB = wumpus world rules + observations

α = [1,2] is safe (참) -> 기존 KB와 새로관측된 knowledge base에 추가되면서 지속적으로 업데이트 된다!

α = [2,2] is safe (거짓)

$KB \Rightarrow_i \alpha$: sentence α can be derived(entailed) from KB by inference algorithm i.

KB $\Rightarrow_i \alpha$: 문장 α 는 추론 알고리즘 i에 의해 KB로부터 유도(entail)될 수 있다

추론 알고리즘 i를 사용해서 α 를 도출할 수 있다!

특성 2가지!

Soundness

whenever $KB \Rightarrow_i \alpha$, it is also true that $KB \Rightarrow \alpha$

알고리즘 i가 KB로부터 α 를 도출했다면, α 는 실제로도 KB에 의해 entail되어야 한다.

알고리즘이 참이라고 하면, 그건 실제로도 무조건 참이다."

Completeness

whenever $KB \Rightarrow \alpha$, it is also true that $KB \Rightarrow_i \alpha$

α 가 실제로 KB에 의해 entail된다면, 알고리즘 i도 α 를 도출할 수 있어야 한다

실제로 참인 것이라면, 알고리즘이 반드시 그것을 찾아내야 한다

Propositional Logic 명제 논리

Syntax

: 가장 간단한 logic

The proposition symbols **P1, P2** are **sentences**.

If **S** is a sentence, **$\neg S$** is a sentence (negation)

If S1 and S2 are sentences,

S1 $\wedge$ S2 is a sentence (conjunction)

논리합 AND

S1 $\vee$ S2 is a sentence (disjunction)

논리곱 Or

S1 $\Rightarrow$ S2 is a sentence (implication)

IF S1이면 S2이다

S1 $\Leftrightarrow$ S2 is a sentence (biconditional)

IFF 필요충분조건.(동치)

Semantics

Specifies true/false for each proposition symbol.

각 명제 기호에 대해 참/거짓을 지정한다

S1 = true

S2 = true

S3 = false

S1 is true and S2 is true iff S1 $\wedge$ S2 is true

S1이 참이고 S2가 참일 때 그리고 그럴 때에만, S1 $\wedge$ S2 가 참이다

진리표에 기반하여 T/F를 구할 수 있다.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Wumpus World Sentences

Let $P(i,j)$ be true if there is a pit in $[i,j]$.

$[i,j]$ 칸에 pit(구덩이)이 있으면 $P(i,j)$ 를 참이라고 하자

Let $B(i,j)$ be true if there is a breeze in $[i,j]$.

$[i,j]$ 칸에 breeze(바람)가 있으면 $B(i,j)$ 를 참이라고 하자

$\neg P(1,1)$ $[1,1]$ 에 pit가 없다

$\neg B(1,1)$ $[1,1]$ 에서 breeze가 느껴지지 않는다

$B(2,1)$ $[2,1]$ 에서는 breeze가 느껴진다

Pits cause breezes in adjacent squares 구덩이는 인접한 칸들에 breeze를 발생시킨다

$B(1,1) \Leftrightarrow P(1,2) \vee P(2,1)$ 양방향이면 참이어야 함. $\vee$ 는 OR

$B(2,1) \Leftrightarrow P(1,1) \vee P(2,2) \vee P(3,1)$ $[2,1]$ 에 breeze가 있다는 것은
 $[1,1], [2,2], [3,1]$ 중 적어도 하나에 pit가 있다는 뜻

Enumeration Algorithm 조합(나열) 알고리즘에 의한 추론

DFS로 열거(enumeration)하는 방법은 sound하고 complete하다

n에 대해서 2^n -> NP-complete 문제이다

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	false	false	false	false	false	false	true	true	true	true	false	false
false	false	false	false	false	false	true	true	true	false	true	false	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	true	false	false	false	false	false	true	true	false	true	true	false
false	true	false	false	false	false	true	true	true	true	true	true	true
false	true	false	false	false	true	false	true	true	true	true	true	true
false	true	false	false	false	true	true	true	true	true	true	true	true
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	true	true	true	true	true	true	false	true	true	false	true	false

6-3

Forward and Backward Chaining

KB는 Horn clause들의 conjunction(AND)이다

- Horn Clause =>
1. 하나의 명제 기호이거나,
 2. (기호들의 conjunction) $\Rightarrow$ 기호 형태이다

-> 각 sentence는 다 AND로 이루어져 있어야 한다. 각 sentence 안은 아래와 같이 가능.

- E.g., $C \wedge (B \Rightarrow A) \wedge (C \wedge D \Rightarrow B)$

Modus Ponens : complete for Horn KBs. Horn KB에 대해서는 complete하다

Horn 형식의 KB에서는 Modus Ponens만으로도 필요한 결론을 빠짐없이 도출할 수 있다

▼ FACT

$\alpha_1, \dots, \alpha_n,$

▼ Horn Clause

$\alpha_1 \wedge \dots \wedge \alpha_n \Rightarrow \beta$

β

α_1 부터 α_n 까지가 모두 사실이고, " $\alpha_1 \wedge \dots \wedge \alpha_n$ 이면 β "라는 규칙도 있으면, β 를 결론으로 낼 수 있다

-> forward chaining이나 backward chaining에 사용할 수 있다

-> 이러한 추론 알고리즘들은 매우 자연스럽게, 선형 시간에 동작한다

Forward Chaining

사실에서 시작 -> 모든 결론 생성

horn clause를 따라감.

FC
Inf: M.P.
KB: Horn

Idea: fire any rule whose premises are satisfied in the KB, add its conclusion to the KB, until query is found.

아이디어: KB 안에서 전제들이 만족되는 규칙이 있으면 그것을 적용하고, 그 결론을 KB에 추가하는 일을 질의(query)를 찾을 때까지 반복한다.

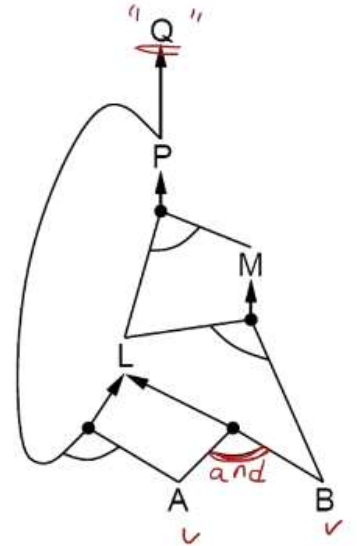
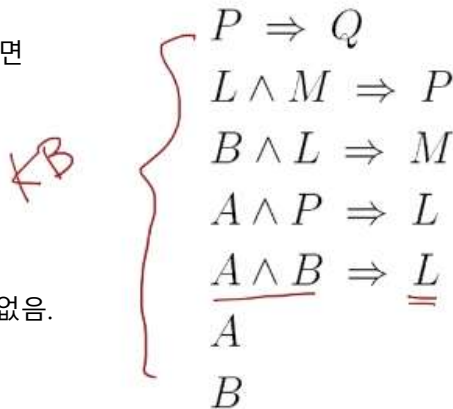
Simple Enumeration과 비교하면

월등하게 빠름

linear하게 찾으면 됨!

참 거짓이지만 비교하면 됨.

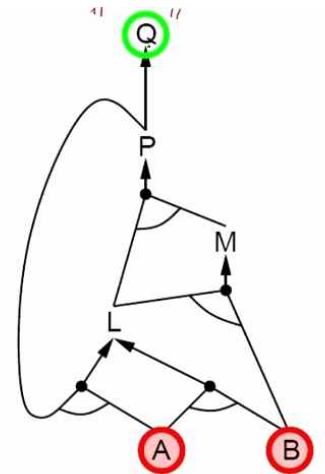
모든 Space를 찾아볼 필요가 없음.



Backward Chaining

결론(Q)에서 시작해서, 그걸 만들려면 뭐가 필요한지 거꾸로 추적하는 방식

Q에서 시작해서 아래로 내려왔다가 다시 위로 올라가는 방식



Forward vs Backward Chaining

Forward chaining: Data-driven; e.g., object recognition. 데이터 주도적.

horn clause를 따라감.

Backward chaining : Goal-driven; appropriate for problem-solving; 목표 주도적

불필요한 조건 검색 X

중간 요약!

Logical agents apply **inference** to a **KB** to derive new information and make decision.

논리적 에이전트는 **KB에 추론(inference)을 적용**해서 새로운 정보를 도출하고 의사결정을 한다

sound할까? complete할까?

-> **Forward Chaining은 complete하다!** KB가 entail하는 결론들을 빠짐없이 찾아낼 수 있다!

Resolution Algorithm

sound하고 complete한 알고리즘이다!

Modus Ponens 대신 CNF를 씀.

Conjunctive Normal Form (CNF) : 리터럴들의 OR($\vee$)를 AND($\wedge$)로 묶은 형태

괄호 밖 AND($\wedge$), 괄호 안 OR($\vee$)

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

$p \Rightarrow q$ 는 $\neg p \vee q$ 로 쓸 수 있음!

제거 규칙

$P \rightarrow Q \equiv \neg P \vee Q$

$P \leftrightarrow Q \equiv (\neg P \vee Q) \wedge (\neg Q \vee P)$

드모르간

$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$

$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$

이중부정

$\neg\neg P \equiv P$

분배

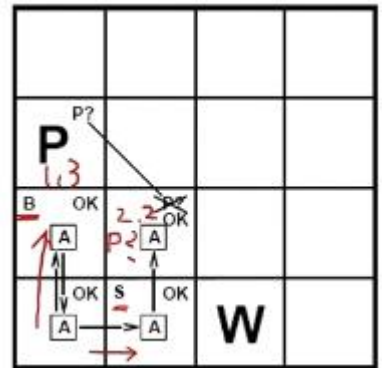
$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$

Resolution inference rule(for CNF) is "sound" and "complete" for propositional logic.
 Resolution은 CNF로 표현된 명제 논리 전체에서 **sound**하고 **complete**하다.

(1,3)또는 (2,2)에 PIT이 있다. (2,2)에 PIT 이 없다

$$\frac{P_{1,3} \vee P_{2,2}, \quad \neg P_{2,2}}{P_{1,3}}$$

(1,3)에 PIT이 있다.

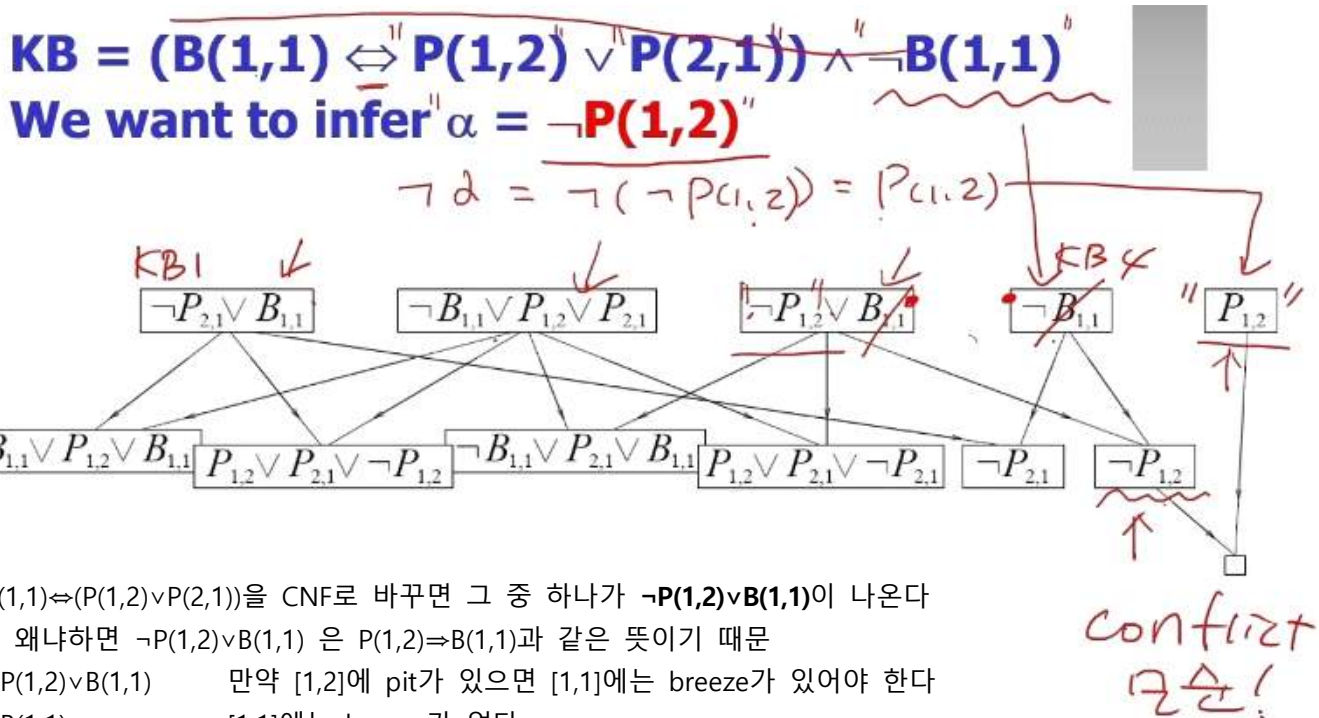


Resolution Algorithm의 실제 동작

Idea : **proof by contradiction** 모순을 이용한 증명

it shows that $KB \wedge \neg \alpha$ is unsatisfiable. $KB \wedge \neg \alpha$ 가 만족 불가능(unsatisfiable)함을 보여준다

-> "α가 맞다는 걸 직접 증명하는 게 아니라, $\neg \alpha$ 가 틀렸다는 걸 보여준다"



1. $B(1,1) \Leftrightarrow (P(1,2) \vee P(2,1))$ 을 CNF로 바꾸면 그 중 하나가 $\neg P(1,2) \vee B(1,1)$ 이 나온다
 왜냐하면 $\neg P(1,2) \vee B(1,1)$ 은 $P(1,2) \Rightarrow B(1,1)$ 과 같은 뜻이기 때문
2. $\neg P(1,2) \vee B(1,1)$ 만약 [1,2]에 pit가 있으면 [1,1]에는 breeze가 있어야 한다
3. $\neg B(1,1)$ [1,1]에는 breeze가 없다
4. 를 뒤집어서 $P(1,2)$ 를 만든다. -> $P(1,2)$ **KB 5개 만들어짐. KB는 사실이다!**

$\neg P(1,2) \vee B(1,1)$ 와 $\neg B(1,1)$ 을 보면 $B(1,1)$ 과 $\neg B(1,1)$ 이 상충된다. -> $\neg P(1,2)$ 만 남는다.
P(1,2)이라고 가정한 것과 상충되어 모순이 일어난다! -> 뒤집은게 모순이므로 $P(1,2)$ 는 참이다!

정리!

Logical Agents

: apply INFERENCE ALGORITHM to KB to derive NEW info. KB에 추론 알고리즘을 적용해서 새로운 정보를 도출

Inference Algorithm

1. Forward/Backward Chaining Algorithm : **Propositional Logic** - Horn Clauses, Modus Ponens
 if $a \Rightarrow b$ and a is true then, b is true (**complete**)
2. Resolution Algorithm: (KB) Propositional Logic - **CNF** (conjunctive normal form)
 needs conversions to CNF proof by contradiction (sound and complete)

KB 예제

법에 따르면, 미국인이 적대적인 국가에 무기를 판매하는 것은 범죄이다.
국가 Nono는 미국의 적국이며, 몇몇 미사일을 가지고 있다.
그리고 그 나라의 모든 미사일은 미국인인 West 대령이 그 나라에 판매한 것이다.
-> West 대령이 범죄자임을 증명하라.

이런 문제는 단일 명제로 표현하기 너무 어렵다...

"서술어, 관계"와 같은 것이 많다. Expression Power가 높다.

-> First Order Logic을 사용함.

Propositional Logic 명제 논리의 장단점

장점

Declarative: pieces of syntax for facts

Compositional: $P(1,2) \vee P(2,1)$

Context-independent

명령적이다 : 사실을 표현하기 위한 문법 조각들을 제공한다

조합 가능하다: $P(1,2) \vee P(2,1)$ 과 같이 수학적으로 조합 가능

문맥 독립적이다. 기호는 하나의 의미를 가지고 있음.

단점

Limited expressive power

표현력이 부족하다.

First-order Logic

Whereas propositional logic assumes world contains facts,

명제 논리는 세상이 사실(facts)들로 이루어져 있다고 가정한다.

first-order logic (like natural language) assumes the world contains

1차 술어 논리는 (자연어처럼) 세상이 다음과 같은 것들로 이루어져 있다고 가정한다.

Objects 객체 : 사람, 집, 숫자, 색깔...

Relations 관계 : 빨강다, 둥글다, ~보다 크다...

Functions 함수 : ~의 아빠, 가장 친한 친구...

실제 술어로 표현을 한다 **Predicate Logic!**

Syntax

상수 : KingJohn, 2, CAU ...

술어(Predicates) : 's Brother, > ...

함수 : sqrt(), LeftLegOf()....

변수 : x, y, a, b

논리, 부등식 : $\wedge$, $\vee$, $\Rightarrow$, = ...

Quantifiers : $\forall$ (모든), $\exists$ (어떤)

"1st order" : 두 대상을 술어로 설명하는 것

"2nd order" : 세 대상을 술어로 설명하는 것... 3rd나 더 높은 것들도 있음.

Atomic Sentences

Atomic Sentences = 술어(항₁, ..., 항_n) 또는 항₁ = 항₂

Term(항) =

e.g. Brother(KingJohn, RichardTheLionheart)

$S1 \wedge S2, S1 \vee S2, S1 \Rightarrow S2, S1 \Leftrightarrow S2$

Complex Sentences 복합문

오른쪽과 같은 형태로 쓸 수 있다. ----->

E.g.,

Brother(KingJohn, Richard)

$\Rightarrow$

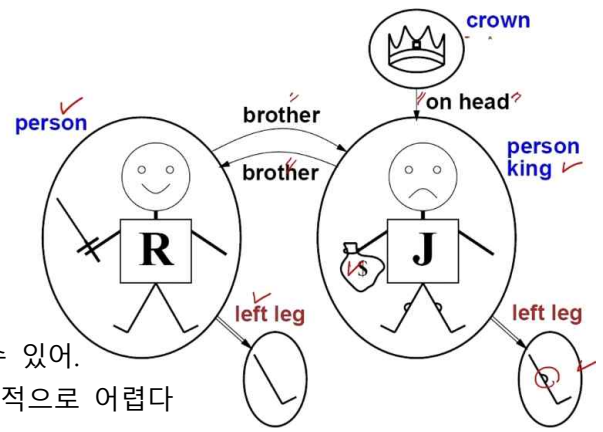
Sibling(Richard, KingJohn)

First-order Logic e.g.

R, J, crown, legs 같은 대상(object)들이 있고
 person, king 같은 속성(property)이 있고
 brother, on head 같은 관계(relation)가 있다

명제논리에서의 entailment는 모델들을
 "열거(enumerate)"해서 계산할 수 있다.

즉, FOL도 "가능한 모델들을 다 따져보자"라는 생각 자체는 할 수 있어.
 핵심은 명제논리에서는 가능한데, FOL에서는 너무 복잡해서 현실적으로 어렵다



Universal Quantification 한정자

$\forall$ <variable> <sentence>

→ 모든 변수값에 대해, 뒤의 문장이 성립한다

버클리에는 있는 모든 사람은 똑똑하다: $\forall x \text{ At}(x, \text{Berkeley}) \Rightarrow \text{Smart}(x)$

-> $\forall x P$ 가 참이라는 것은, 모델 안의 가능한 모든 객체를 x에 넣었을 때 P가 참이라는 뜻이다.

$\text{At}(\text{KingJohn}, \text{Berkeley}) \Rightarrow \text{Smart}(\text{KingJohn}) \wedge \text{At}(\text{Richard}, \text{Berkeley}) \Rightarrow \text{Smart}(\text{Richard})$

$\text{At}(\text{KingJohn}, \text{Berkeley})$ 이면 $\text{Smart}(\text{KingJohn})$ 이고, $\text{At}(\text{Richard}, \text{Berkeley})$ 이면 $\text{Smart}(\text{Richard})$ 이고, ...

많이 틀리는 포인트는 $\forall x$ 뒤에 $\wedge$ 를 중심으로 문장을 만드는 것이다.
 그래서 전칭정량에서는 보통 $\wedge$ 보다 $\Rightarrow$ 가 더 자연스럽다.

$\forall x \text{ At}(x, \text{Berkeley}) \wedge \text{Smart}(x)$ X
 $\forall x (\text{At}(x, \text{Berkeley}) \Rightarrow \text{Smart}(x))$ O

$\exists$ <variable> <sentence>

→ "어떤 것이 하나는 있다" 는 뜻이야.

스탠퍼드에 있는 누군가는 똑똑하다: $\exists x \text{ At}(x, \text{Stanford}) \wedge \text{Smart}(x)$

O

많이 틀리는 포인트는 $\forall x$ 뒤에 $\Rightarrow$ 를 중심으로 문장을 만드는 것이다. $\exists x \text{ At}(x, \text{Berkeley}) \Rightarrow \text{Smart}(x)$

X

Quantifier의 특징

순서 바뀌도 되는 것

$$\forall x \forall y \leftrightarrow \forall y \forall x$$

$$\exists x \exists y \leftrightarrow \exists y \exists x$$

순서를 바꾸면 안 되는 것

$$\exists x \forall y \neq \forall y \exists x$$

e.g, $\exists x \forall y \text{ Loves}(x, y)$

어떤 한 사람 x가 있어서, 모든 y를 사랑한다.

한 명이 전부를 다 사랑하는 경우야.

e.g, $\forall y \exists x \text{ Loves}(x, y)$

모든 y에 대해, 그 y를 사랑하는 어떤 x가 존재한다.

Sentences

$\forall x, y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$ 모든 x, y에 대해, x가 y의 형제이면 x와 y는 형제자매이다

$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow \text{Sibling}(y, x)$ 모든 x, y에 대해, x와 y가 형제자매 관계이면 y와 x도 형제자매 관계이다

$\forall x, y \text{ Mother}(x, y) \Leftrightarrow \text{Female}(x) \wedge \text{Parent}(x, y)$ 누군가의 어머니는 그 사람의 여성 부모이다

Substitution 대입, 할당

Yes, {a/Shoot} (substitution)

Shoot를 a에 넣어라, 할당해라

Wumpus World에서

$\forall b, g, t \text{ Percept}([Smell, b, g], t) \Rightarrow Smelt(t)$

모든 b, g, t에 대해, 시각 t에서 지각이 [Smell, b, g]이면 Smelt(t)

$\forall s, b, t \text{ Percept}([s, b, Glitter], t) \Rightarrow AtGold(t)$

모든 s, b, t에 대해, 시각 t에서 지각이 [s, b, Glitter]이면 AtGold(t) 이다

-> $\forall t \text{ AtGold}(t) \Rightarrow \text{Action}(\text{Grab}, t)$

반사 규칙! Reflex

-> $\forall t \text{ AtGold}(t) \wedge \neg \text{Holding}(\text{Gold}, t) \Rightarrow \text{Action}(\text{Grab}, t)$

이미 금 들고 있으면 또 Grab 하지 않게 개선

숨겨진 성질 추론하기

“Breezy” 술어의 정의: $\forall y \text{ Breezy}(y) \Leftrightarrow \exists x \text{ Pit}(x) \wedge \text{Adjacent}(x, y)$

Diagnostic rule 진단 규칙 : $\forall y \text{ Breezy}(y) \Rightarrow \exists x \text{ Pit}(x) \wedge \text{Adjacent}(x, y)$

원인으로부터 결과 추론

Casual rule 인과 규칙 : $\forall x, y \text{ Pit}(x) \wedge \text{Adjacent}(x, y) \Rightarrow \text{Breezy}(y)$

결과로부터 원인 추론

Planning

At(Agent, [1,1], S0)

#초기 상태 S0에서 에이전트는 [1,1]에 있다

At(Gold, [1,2], S0)

#초기 상태 S0에서 금은 [1,2]에 있다

Query: ASK(KB, $\exists s \text{ Holding}(\text{Gold}, s)$)

#어떤 상태 s가 존재해서 그 상태에서 금을 들고 있는가?

금을 손에 넣는 상태에 도달할 수 있나

Answer : {s / Result(Grab, Result(Forward, S0))}

#즉 계획은 Forward 하고, 그 다음 Grab 이라는 뜻

Inference algorithm for First-order Logic 명제논리 추론으로의 환원

KB가 다음만 포함하고 있다고 하자:

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

모든 x에 대해, x가 왕이고 탐욕스러우면 x는 사악하다

King(John), Greedy(John)

John은 왕이고, John은 탐욕스럽다

Brother(Richard, John)

Richard는 John의 형제이다

문장을 가능한 모든 방식으로 인스턴스화하면:

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

가능한 방식 다 해보기

John은 왕이고, John은 탐욕스럽다

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

Richard가 왕이고 탐욕스러우면 Richard는 사악하다

King(John), Greedy(John)

John은 왕이다, John은 탐욕스럽다

Brother(Richard, John)

Richard는 John의 형제이다

새로운 proposition symbols들을 만들고 + resolution를 적용한다

-> Propositionalization, 불필요한 정보가 생겨남.

Unification

~~ substitution 대입과 같다.

GMP : Generalized Modus Ponens

First-order Logic에서도 Modus 쓸 수 있음.

$$\frac{p_1', p_2', \dots, p_n', (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{q\theta}$$

변수가 들어있어서 Generalized! 라고 표현

---> 이렇게 쓸 수는 있는데 비효율적이다!

p_1' is *King(John)* p_1 is *King(x)*
 p_2' is *Greedy(y)* p_2 is *Greedy(x)*
 θ is $\{x/\text{John}, y/\text{John}\}$ q is *Evil(x)*
 $q\theta$ is *Evil(John)*

이제까지 KB가 First-order Logic 인 것을 살펴봤다.

-> Inference는 3가지 방식으로 가능하다.

1. Forward Chaining
2. Backward Chaining
3. Resolution

$\{1, 2\}$ Forward/B.
 $\uparrow$ Resolution
 KB: FOL

• A crime for an American to sell weapons to hostile nations:

American(x) $\wedge$ Weapon(y) $\wedge$ Sells(x,y,z) $\wedge$ Hostile(z) $\Rightarrow$ Criminal(x)

• Nono ... has some missiles:

Owens(Nono, M1) and Missile(M1)

• ... all of its missiles were sold to it by C. West:

$\forall x$ Missile(x) $\wedge$ Owens(Nono,x) $\Rightarrow$ Sells(West,x,Nono)

• Missiles are weapons:

Missile(x) $\Rightarrow$ Weapon(x)

• An enemy of America counts as "hostile":

Enemy(x, America) $\Rightarrow$ Hostile(x)

• West, who is America ...:

American(West) $\leftarrow$

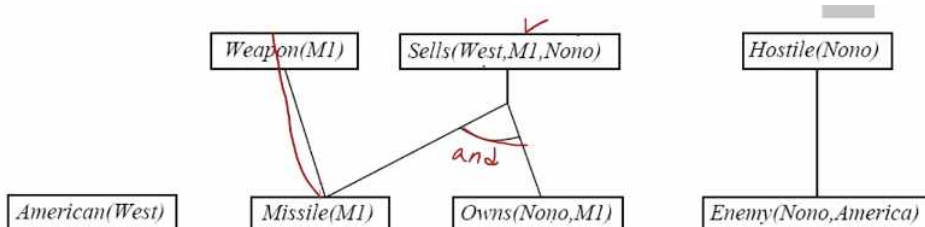
• The country Nono, an enemy of America ...:

Enemy(Nono, America) $\leftarrow$

1. Forward Chaining

Imcomplete 불완전한 방법이다 -> 무한 루프에 빠질 수 있다.

Horn ~에서는 완전한 방법이지만, 변수가 있는 First-order Logic에서는 불완전



• Missiles are weapons:

Missile(x) $\Rightarrow$ Weapon(x) $\leftarrow$ True

• ... all of its missiles were sold to it by C. West:

$\forall x$ Missile(x) $\wedge$ Owens(Nono,x) $\Rightarrow$ Sells(West,x,Nono) $\leftarrow$ True

• An enemy of America counts as "hostile":

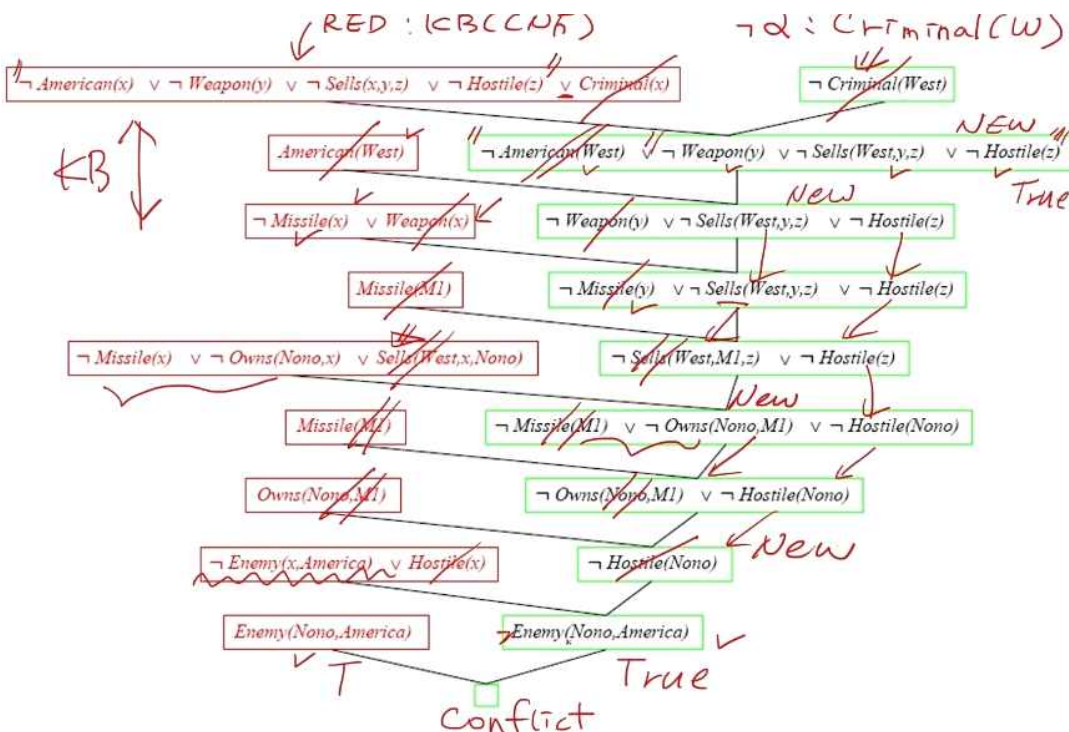
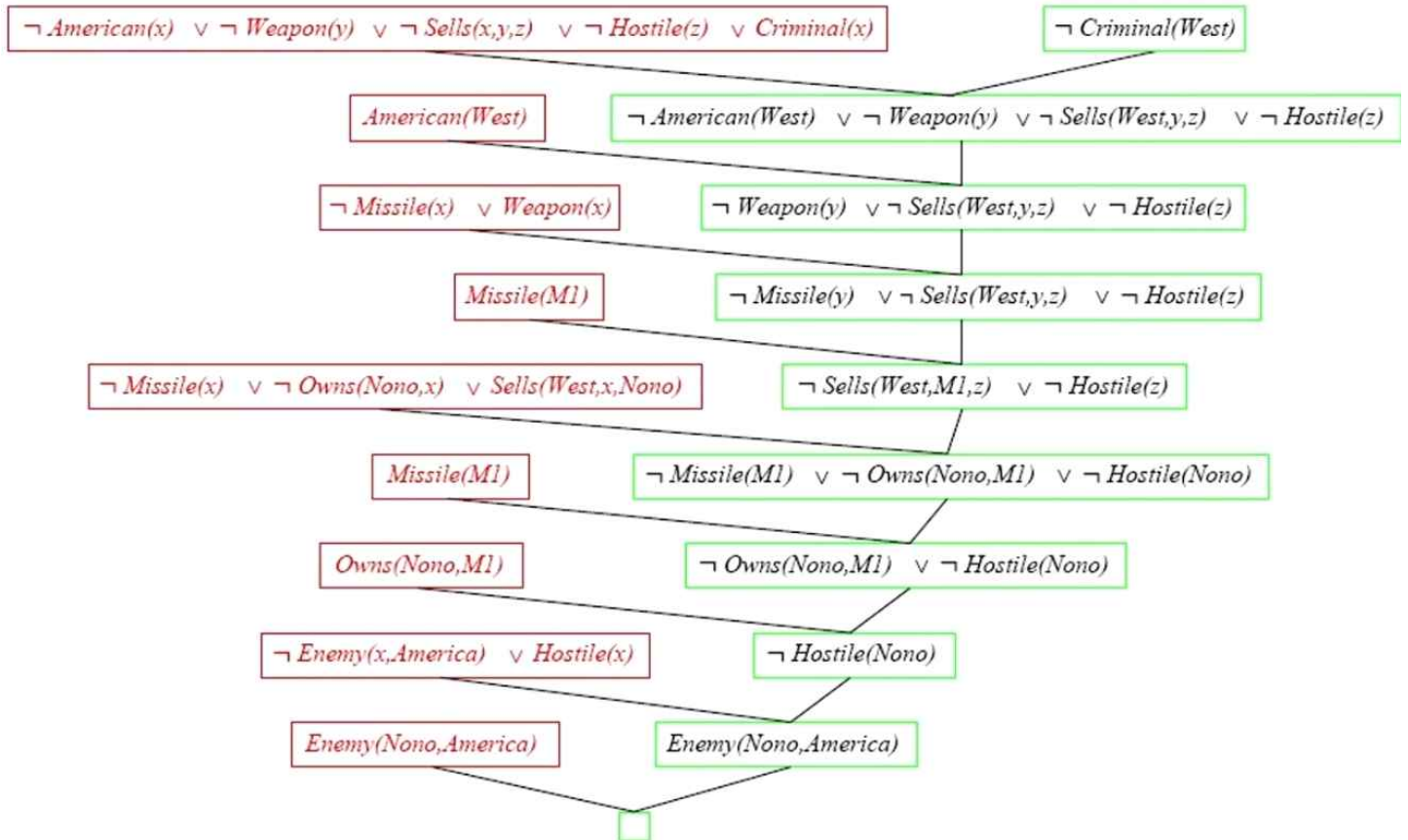
Enemy(x, America) $\Rightarrow$ Hostile(x)

$$\theta = \{x/Ken\}$$

$$\frac{\neg Rich(x) \vee Unhappy(x)}{Rich(Ken)} \quad Unhappy(Ken)$$

FACT중에 모순되는게 나오면 지워버린다!
일단 모두 CNF로 바꾼다

- "KB" $\rightarrow$ "CNF"
- A crime for an American to sell weapons to hostile nations:
 $American(x) \wedge Weapon(y) \wedge Sells(x,y,z) \wedge Hostile(z) \Rightarrow Criminal(x)$
 - Nono ... has some missiles:
 $Owns(Nono, M1) \wedge Missile(M1)$
 - ... all of its missiles were sold to it by C. West:
 $\forall x Missile(x) \wedge Owns(Nono,x) \Rightarrow Sells(West,x,Nono)$
 - Missiles are weapons:
 $Missile(x) \Rightarrow Weapon(x)$
 - An enemy of America counts as "hostile":
 $Enemy(x,America) \Rightarrow Hostile(x)$
- "Natural"*
- $P \Rightarrow Q \equiv \neg P \vee Q$
 $(CNF) \quad \neg M \vee W$



Artificial Intelligence 용어(term) 최초 사용

Dartmouth Conference (1956년)

AI Wave Boom

1st Boom

50s 후반-60s

1. AI를 사용한 문제해결 : 탐색, inference algorithm
2. MIT, Stanford, CMU 등에서 연구...
search algorithm을 사용해서 Toy 프로젝트 함.
Tic-Tac-Toe, Chess 도전 : 탐색 알고리즘 개선작업
3. Winter :1970s

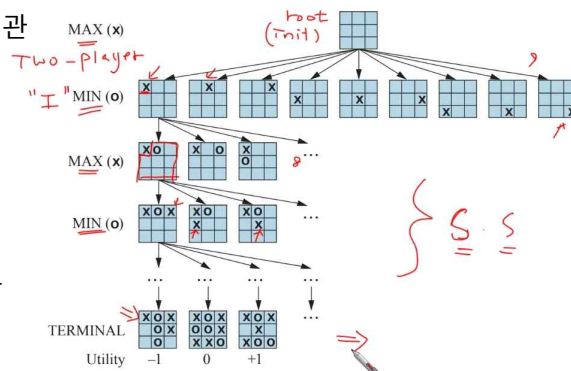
USA -> ALPAC의 보고서 : 항공기 매뉴얼을 자동번역 할 수 있지 않을까? -> 비판, 투자심리 위축
당시의 작은 토이문제들은 Real-life 문제가 아니었음. 실생활과 무관

Tic-Tac-Toe : 한줄 먼저 놓으면 이기는 게임

-> 모든 트리를 그리기가 너무 복잡했음.

Games and Search Problems

1. 일반적인 탐색에서는 optimal solution이란 초기 ->목표로 가는 일련의 이동 순서를 뜻함.
2. **GAME search**에서는 상대가 할 수 있는 모든 가능한 행동에 대해서 가장 좋은 수를 찾아야 한다
→ "적대적 탐색(adversarial search)" + "예측 불가능성" + 시간 제한
=> optimal 대신 approximate solution을 찾는 것으로 하향함.



2nd Boom - 전문가 시스템

Q) 트리에서 모든 path를 다 가야할까?

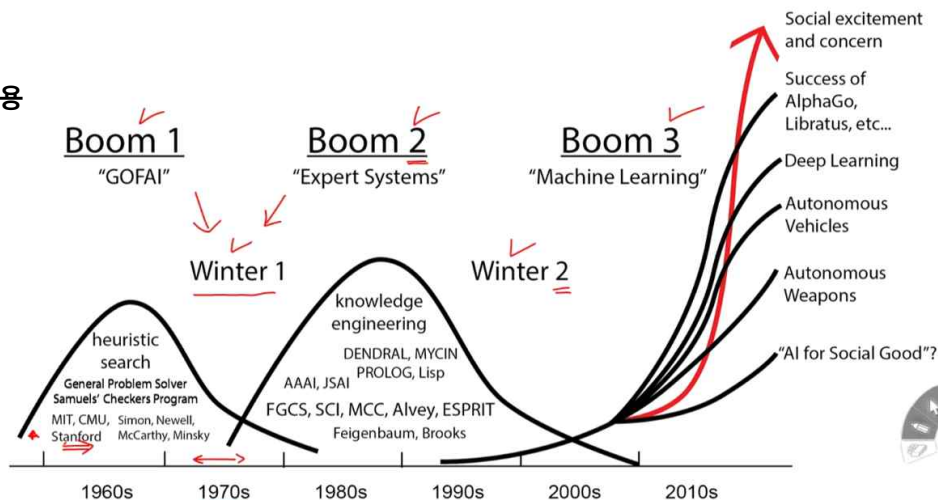
- A) B&B pruning을 하자 -> game으로 오면서 **Alpha-Beta Pruning**으로 발전하게 됨
Alpha 변수 = MAX 플레이어 입장에서 지금까지 발견한 값 중에 제일 좋은 값
Beta 변수 = MIN 플레이어 입장에서 지금까지 발견한 값 중에 제일 좋은 값

evalutaion func : $f = g + h$

Alpha-Beta Pruning은 지속적으로 값을 업데이트를 하면서 남아있거나 불필요한 branch를 prune했다.

2nd Boom : 실생활의 "knowledge"를 사용하자!

MIT 1964 "**Eliza**" : 초기 대화형 시스템. 가짜...



Stanfords 1970 : MYCIN 마이신 - Exper System 전문가 시스템

전염성 혈액진단 환자에 대한 진단, 항생제 처방

약 600개의 규칙 데이터베이스 KB : If ~ then ~

KB : IF 세균 모양이 막대 모양 AND 환자 고통이 극심하다
THEN 균은 "녹농균"이다

처방진단 정확도 : 69% 의사는 80%

cf) 현시점 CT/MRI 기반의 딥러닝 진단율 : 60-70%/80%

왜 다시 겨울이 왔을까? Limits 가 있었다.

1. 전문가의 지식을 획득하는 것이 어려웠음. 비용이 많이 들었음. 규모도 작음
2. 지식의 수, 규칙의 폭발적 증가

-> KB의 consistency 이슈가 생김. 충돌이 생기지 않게 하기가 어려움

-> Knowledge의 표현과 관리에 이슈가 생김

e.g. 어제 위 주변들이 메스하더라고요.. -> 애매한 표현은 define하기가 어렵다.

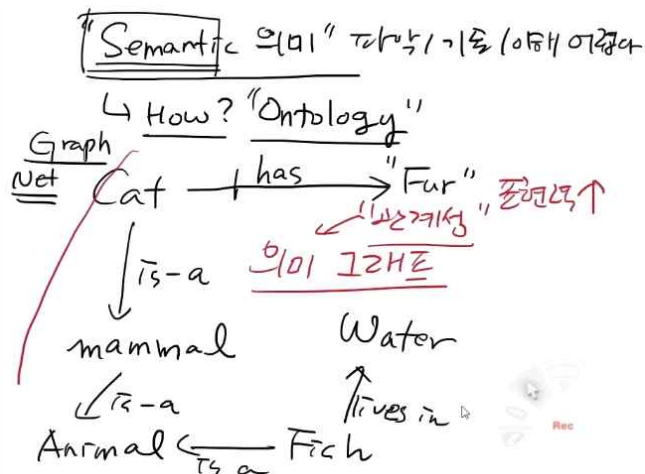
-> 이후에 아이디어를 내게 됨.

상식 수준의 지식을 기술하기 어렵구나..

새로운 연구분야 개척하는 근본 연구가 진행됨

Semantic(의미) 파악/기술/이해가 어렵구나

-> 지식 표현 방법 Ontology(온톨로지) 개발!



Heavy-weight Ontology : 인간이 적극 개입하여 관계성을 찾는 것

Light-weight Ontology : 현실적 대안. Data-Driven. 완전하진 않음.

2차 겨울 : 1987-1995

인공지능 AI 언급X (투자가 안되기 시작함)

-> 다른 용어 사용. "머신러닝", "패턴인식"

1차 봄 이후 시대 이후로 지속적 개선, 발전되고 있음.

fundamental research :

1. machine learning(1차 봄. 신경망), 패턴인식, 데이터마이닝

학습=분류=인식

2. 웹 검색 엔진/알고리즘

3. 자연어 처리(NLP), 기계 번역

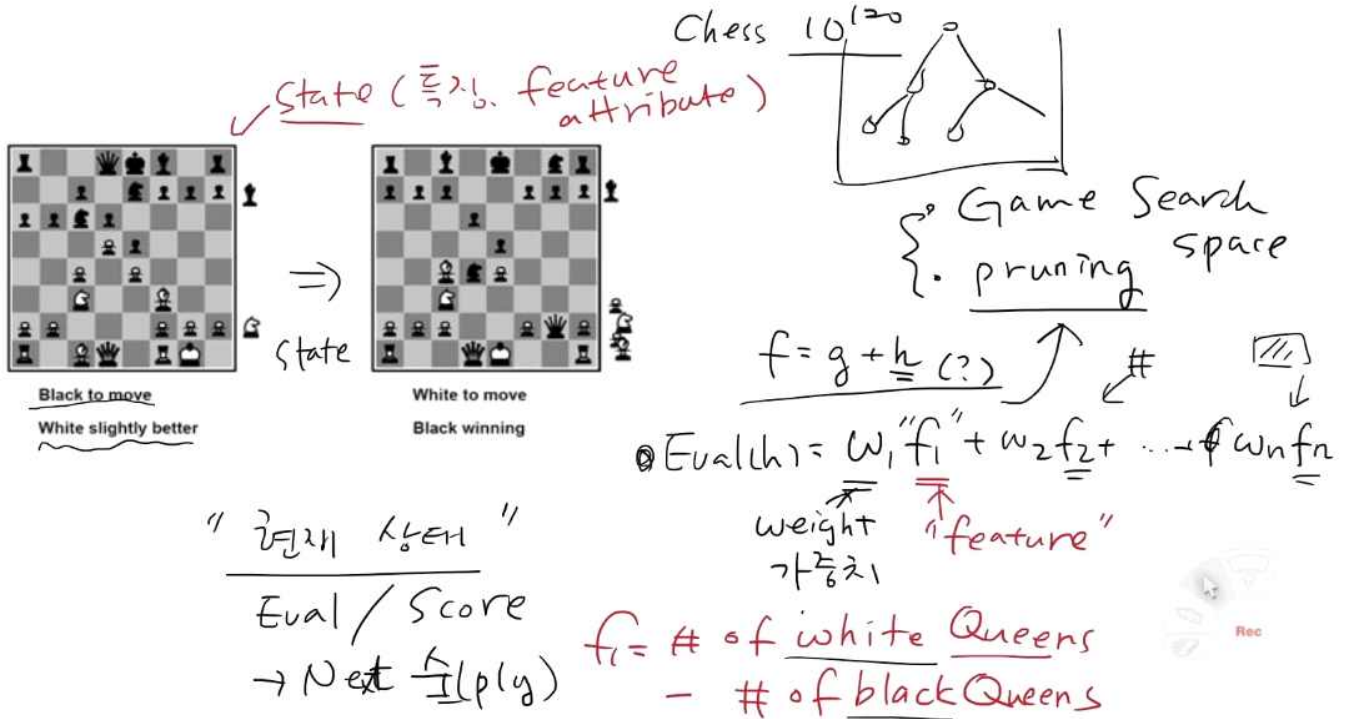
기존의 추론.지식 표현과 다르게, 확률적/통계적/선형적 이론으로 분석하고 활용하는 연구를 많이 진행

3rd boom - Machine Learning

- 환경 변화 : Data의 폭발적 증가 (디지털시대의 도래)
HW의 발전(CPU, GPU, 병렬머신), Global IT 기업(vs 대학)
-> 산업 IT혁명 AI+Industry
----> existing algorithms(AI) 실생활에 적용이 되기 시작함.

1997 IBM 딥블루 : vs Gary Kasparov (6경기 매치)

체스 복잡도 : 10^{120} (기존 머신은 이기기가 어려움)



딥블루는 병렬 컴퓨팅을 해서 Brute Force로 계산함
Search Algorithm + eval. function + machine의 성능 향상

2016년 DeepMind AlphaGo

- 바둑 : AI에서 가장 도전적 게임 ($361! = 10^{171}$)
-> Advanced search algorithm + Deep NNs(deep q-network)
-> HW 발전(초당 몇 억수 계산), 중반 이후 바둑 돌의 위치가 fixed,
시간이 지날수록 유효한 바둑돌의 개수가 한정 : search space가 급격히 줄어들음 -> 이세돌 인터뷰
-> 몬테카를로법 : evaluation function 개선
----> 미래에 예측. 남아 있는 바둑돌의 상태공간 : 남은 수에 대해서 랜덤하게 p1, p2 진행(playout)
100회 진행 -> 70:30 점수는 70점 (HW 성능이 뒷받침이 되어야 함)

2011년 IBM Watson

Eliza -> 마이신 -> 언어대화시스템 발전 -> IBM 왓슨!
질문-응답 시스템 Question-Answering System

1997년 이후 1997 퀴즈쇼를 푸는 컴퓨팅 AI 도전결심
초기에는 위키피디아 -> light-weight ontology 구축 생성 시도
질문의 의미(Semantic) 이해하고 답을 하는것이 아니라, 질문 중 keyword와 관련성이 높은 답을 탐색해서 제출
-> 현재는 NLP, QA 연구를 선도함. 국방, 의료 등 다방면 활용 중
-> Siri, 챗봇 등 연구 서비스를 창출.

인공지능이란?

- ex) 비행기 제작 : 날개짓하는 비행기를 만들려고 노력했지만, 결국에는 실패함.
-> 무언가를 만들 때 반드시 해당 사물과 똑같이 할 필요는 없다.
-> 인공지능도 마찬가지. 원리를 찾고, 컴퓨터로 실현하면 된다!

Q1. 컴퓨터가 의미를 이해할 수 있을까? 구글 검색은 의미를 이해할까?

Q2. The Frame Problem

Q3. Symbol Grounding Problem

Q4. Chines room Argument

Q5. Strong AI vs Weak AI

Further Issue

1. Semantic Ambiguity Problem

핵심:

하나의 문장이 문맥에 따라 여러 뜻으로 해석될 수 있다는 문제.

예시:

He saw a woman in the garden with a telescope.

→ 망원경으로 본 건지, 여자가 망원경을 가진 건지 애매함.

중요한 점:

- 인간은 경험, 상식, 문맥으로 뜻을 잘 판단함
- AI/NLP는 이런 의미 해석이 어려움

2. The Frame Problem

핵심:

AI가 어떤 행동을 할 때, 세상에서 무엇이 바뀌고 무엇이 그대로인지 어떻게 골라낼 것인가의 문제.

쉬운 뜻:

행동 하나를 하면 관련 없는 것까지 전부 다시 계산하면 너무 비효율적임.

중요한 점:

- AI는 중요한 변화만 추려서 생각해야 함
- 즉, 무시해도 되는 것과 꼭 봐야 하는 것을 구분해야 함

3. The Symbol Grounding Problem

핵심:

기호(symbol), 단어, 문자가 어떻게 실제 세계의 의미와 연결되는가 하는 문제.

쉬운 뜻:

AI는 단어끼리의 관계는 다룰 수 있어도, 그 단어가 현실에서 무엇을 뜻하는지 직접 경험으로 아는 건 아님.

예시:

'모래'라는 단어 뜻을 사전으로는 설명할 수 있어도, 인간은 실제로 보고, 만지고 의미를 더 잘 이해함.

중요한 점:

- 인간: 경험을 통해 의미 이해
- AI: 기호 사이의 연결은 가능하지만, 실제 체험 기반 의미 이해는 어려움

1. Semantic Ambiguity Problem

언어 표현 하나의 고정된 의미를 가지지 X
문맥에 따라 여러 의미로 해석 O

He saw a woman in the garden with a telescope.

--> 문맥 Context: 인간이 경험과 지식 체계에 의존

--> NLP, LLMs: 근본적 어려움 -> 통계적 머신러닝, 맥락 모델, 세계적 지식이 동원, 유기적 연결.

2. The Frame Problem (by Daniel Dennett)

우리가 또는 로봇이, AI이 어떤 행동을 할 때, 무엇이 변하고, 무엇이 변하지 않는지를 AI가 어떻게 판단하는가?

로봇, 배터리 on 수레 Wagon. 배터리 위에 시한 폭탄.

-> pullout + 폭탄도 같이 가지고 옴.

-> next design: 충분한 추론을 통해. 성공 전략.

-> 과도한 추론 발생: action -> 방의 온도? 색? 바퀴 수? 바닥의 온도, 재질은? 바퀴의 회전수는 달라지는가? 등.

==> 끝없는 '변하지 않는 것'들에 대한 증명 -> 추론도중폭발

==> AI는 중요한 것만 생각하는 능력을 가질 수 있다는 것을 증명하기 어렵다.

-> Frame problem은 AI가 세상에 대해 <알아야 할 것들> 보다는 "무시할 수 있는, 즉 알 필요가 없는 것들" 구별하는 능력을 보유하는 것이 중요!

3. The Symbol Grounding Problem (by Stevan Harnad)

기호symbol, 개념, 문자가 실제 세계의 <의미> 연결?
symbol과 semantic의 연결이 어렵다 -> 주장.

--> '모래' 사전 정의: 잘게 부스러진 돌과 광물 입자.

--> 모래 -> 유치원생에게 직접 만져보게 한다.(embodiment)

인간은 보고, 만지고, 맛보며 ... <의미>를 알아갑니다.

AI는 기호-기호간의 관계, 실제적 경험, feeling을 통한 의미 획득과 이해가 가능하지 않다. I

4. Chinese Room Argument

핵심:

AI는 기호를 규칙대로 처리할 수는 있지만,
그것이 진짜 의미 이해를 뜻하는 것은 아니라는 주장.

쉬운 뜻:

겉으로는 정답을 잘 말해도,
속으로 정말 이해하고 있는 건 아닐 수 있다는 거야.

중요한 점:

- 튜링 테스트에 대한 비판으로 자주 나옴
- “잘 대답함 = 이해함”은 아니다
- 그래서 계산이 곧 이해인가? 라는 질문이 생김

4. Chinese Room Argument 중국어 방 논증

: 컴퓨터 AI는 기호를 조작할 수 있지만, 의미를 이해할 순 없다.
튜링 테스트를 비판.

-> 정답을 말하는 것과 이해를 하는 것은 다르다.

->> Strong AI에 대한 강한 회의 : 계산이 이해인가?

5. Strong AI vs. Weak AI

Weak AI

핵심:

AI는 특정 작업을 잘할 수 있지만,
진짜 의식·이해·마음이 있다고 보지는 않음.

쉽게 말하면:

- 계산 잘함
- 패턴 인식 잘함
- 문제 해결 잘함
- 하지만 마음을 가진 건 아님

5. Strong AI vs. Weak AI

Weak AI: AI는 인간처럼 이해하지는 않지만, 특정 과제를 잘 수행할 수 있다. -> 도구 AI, 계산, 패턴 인식 추론 AI.
==> 이해, 의식, 마음, 자유의지 X
==> AI는 행동을 흉내낼 수 있지만, 마음을 가지지는 못한다.

Strong AI: 실제로 Mind, understanding, 의식, free will O

-> Q: 여러분의 생각은 어떠한가요? 반드시 생각해보기 바랍니다.

Strong AI

핵심:

AI가 단순한 도구가 아니라,
인간처럼 진짜 이해, 의식, 마음, 자유의지를 가질 수 있다고 보는 입장.

쉽게 말하면:

AI도 언젠가는 정말 생각하고 이해하는 존재가 될 수 있다는 주장.